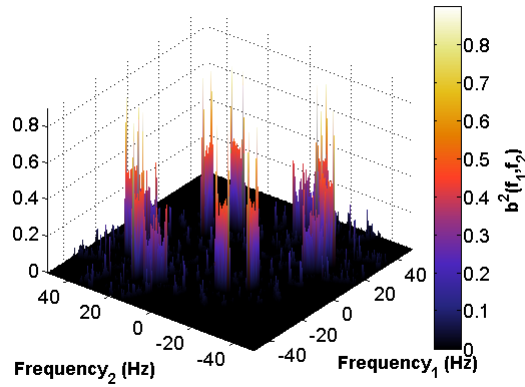


Technical Report

BISPECTRAL ANALYSIS: THEORY & IMPLEMENTATION

For Plasma Physics Laboratory — Koepke (PPL-K)

G. A. Riggs — ©2021 WVU



Contents

1	Introduction	2
2	The N-Spectrum	2
2.1	Fourier analysis	2
2.2	Generalized correlation	3
2.3	N -Coherence	5
3	Implementation	6
3.1	Discretization	6
3.2	Windowing	8
3.3	Bi-frequency space	11
3.4	Phase dynamics	12
3.5	Algorithm	14
3.6	Wavelet analysis*	15
4	Practicum	19
4.1	pplk_bispec manual	19
4.2	Example MATLAB analysis	24
4.3	Source code	28

1 Introduction

This report is meant as a guide to understanding *higher-order spectral methods* in discrete signal processing. While the theoretical underpinning is interesting in its own right, in practice, the merit of these methods is diagnostic. That is, behavior which is characteristic to systems of nonlinearly-coupled oscillators is made manifest. Hence, they are highly applicable to study of plasma fluctuations — or more broadly, nonlinear physics. As this document serves to speak in general terms, we will not focus any one “most-interesting” system, but rather develop the tools to efficiently apply this analysis in any circumstance.

In particular, after quickly reviewing the tenets of Fourier theory, we define the so-called N -spectrum using a generalization of the convolution theorem. Focusing on the special case $N = 3$ (the *bispectrum*), we discuss discretization, then an algorithm for digital implementation. Along the way, we attempt to highlight salient qualities of the bispectrum, while maintaining a lucid, didactic voice in our narrative. Finally, we present a debugged MATLAB routine, complete with user manual and example analysis.

2 The N -Spectrum

2.1 Fourier analysis

Consider an integrable, real-valued function $x : \mathbb{R} \rightarrow \mathbb{R}$ of time t . Furthermore, demand that

$$x(t) = 0 \quad \forall t \notin \mathcal{T}, \quad (2.1)$$

where $\mathcal{T} = \{t : t \in [t_i, t_f]\} \subset \mathbb{R}$ is some proper subset of the real line. That is, we consider a nonsingular function of time which is nonzero only in the interval $t_i \leq t \leq t_f$. Without loss of generality, we effect the transformation $t' \rightarrow t - t_i$ so that $t'_i = 0$ and $t'_f = T \equiv t_f - t_i$. Dropping the primes, we may expand the function $x(t)$ in the following basis:

$$x(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi i n t / T} \quad \forall t \in [0, T] \quad (2.2)$$

which is the well-known *Fourier series* representation.¹ The orthogonality of the basis with respect to the inner product,

$$\int_0^T e^{2\pi i n t / T} e^{-2\pi i m t / T} dt = T \delta_{nm}, \quad (2.3)$$

allows us to quickly deduce the coefficients c_n , via

$$c_n = \frac{1}{T} \int_0^T x(t) e^{-2\pi i n t / T} dt. \quad (2.4)$$

Clearly, $x(t)$ must be bounded on the interval $t \in [0, T]$; however, there is no requirement it be differentiable or even continuous.² Now, it is trivial to effect a symmetric transformation such that³

$$c_n = \frac{1}{T} \int_{-T/2}^{T/2} x(t) e^{-2\pi i n t / T} dt. \quad (2.5)$$

Defining now

$$f = \frac{n}{T}, \quad \text{and} \quad \mathcal{F}(f) = T c_n, \quad (2.6)$$

¹Crucially, this representation satisfies $x(t + T) = x(t)$, and thus does *not equal the original function* when $t \notin [0, T]$.

²Indeed, a sufficient condition for *pointwise* convergence of the Fourier series is differentiability, although the series will generally converge *almost everywhere* in $[0, T]$.

³We are being a bit glib here. Generally, the c_n in (2.4) are different than those in (2.5). In either case, a unique expansion is guaranteed by the orthogonality condition.

we have in the limit $T \rightarrow \infty$,

$$\mathcal{F}\{x\}(f) \equiv \hat{x}(f) = \int_{-\infty}^{\infty} x(t) e^{-2\pi i f t} dt, \quad (2.7)$$

which we might recognize as the *Fourier transform* of $x(t)$. Making use of $dn = Tdf$, the sum (2.2) is converted to an integral,

$$x(t) = \mathcal{F}^{-1}\{\hat{x}\}(t) \equiv \int_{-\infty}^{\infty} \hat{x}(f) e^{2\pi i f t} df, \quad (2.8)$$

which is the *inverse Fourier transform*. Substitution of $\hat{x}(f)$ with (2.7) leads to

$$x(t) = \int_{-\infty}^{\infty} df e^{2\pi i f t} \int_{-\infty}^{\infty} dt' x(t') e^{-2\pi i f t'}. \quad (2.9)$$

As $x(t)$ is integrable by definition, we do not violate Fubini's theorem when exchanging integrations, and hence

$$\delta(t - t') = \int_{-\infty}^{\infty} df e^{-2\pi i f (t - t')}. \quad (2.10)$$

This implies, for example, that $\mathcal{F}\{1\} = \delta(f)$, and⁴

$$\mathcal{F}\{\cos(2\pi f_0 t)\}(f) = \frac{1}{2}[\delta(f - f_0) + \delta(f + f_0)], \quad (2.11)$$

which can be deduced by changing variables in (2.10). Now, we will require the N -dimensional analogues of (2.7) and (2.8), which are

$$\mathcal{F}\{x(t_1, \dots, t_N)\}(f_1, \dots, f_N) = \underbrace{\int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty}}_N x(t_1, \dots, t_N) \prod_{k=1}^N dt_k e^{-2\pi i f_k t_k}, \quad (2.12)$$

and

$$\mathcal{F}^{-1}\{\hat{x}(f_1, \dots, f_N)\}(t_1, \dots, t_N) = \underbrace{\int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty}}_N \hat{x}(f_1, \dots, f_N) \prod_{k=1}^N df_k e^{2\pi i f_k t_k}, \quad (2.13)$$

respectively. We see that if $x(t_1, \dots, t_N)$ is separable into a product of N functions of a single variable, then the multidimensional Fourier transform is simply the product of the individual (one-dimensional) transforms.

2.2 Generalized correlation

Given a sequence of $N > 1$ real-valued, integrable functions $\{x_k\}_{k=1}^N$, where $x_k : \mathbb{R} \rightarrow \mathbb{R}$, we define their *N -correlation* by

$$\begin{aligned} C_{x_1 \dots x_N}(t_1, \dots, t_{N-1}) &= \int_{-\infty}^{\infty} d\tau x_1(\tau) x_2(t_1 + \tau) \dots x_N(t_{N-1} + \tau), \\ &= \int_{-\infty}^{\infty} d\tau x_1(\tau) \prod_{k=1}^{N-1} x_{k+1}(t_k + \tau). \end{aligned} \quad (2.14)$$

For clarity, suppose $N = 2$. Then we have the *cross-correlation*,

$$C_{x_1 x_2}(t) = \int_{-\infty}^{\infty} d\tau x_1(\tau) x_2(t + \tau). \quad (2.15)$$

⁴More technically, we recognize that (2.10) is valid in the sense of *distributions*. It is straightforward to show that the transform of a rectangular function of height unity and width T [e.g., $H(t + T/2) - H(t - T/2)$] is equal to $\sin(\pi T f)/\pi f$. In the limit $T \rightarrow \infty$, we formally recover the delta function $\delta(f)$.

To proceed, we use (2.8) to expand the second term in the integral:

$$\begin{aligned} C_{x_1 x_2}(t) &= \int_{-\infty}^{\infty} d\tau x_1(\tau) \int_{-\infty}^{\infty} df \hat{x}_2(f) e^{2\pi i f(t+\tau)}, \\ &= \int_{-\infty}^{\infty} df \hat{x}_2(f) e^{2\pi i f t} \int_{-\infty}^{\infty} d\tau x_1(\tau) e^{-2\pi i(-f)\tau}, \\ &= \int_{-\infty}^{\infty} df \overline{\hat{x}_1(f)} \hat{x}_2(f) e^{2\pi i f t}, \end{aligned} \quad (2.16)$$

where we have noticed $\hat{x}(-f) = \overline{\hat{x}(f)}$ for real-valued functions ($\bar{x} = x$). Thus,

$$C_{x_1 x_2}(t) = \mathcal{F}^{-1} \left\{ \overline{\hat{x}_1(f)} \hat{x}_2(f) \right\}. \quad (2.17)$$

As $\mathcal{F}\mathcal{F}^{-1} = \mathcal{F}^{-1}\mathcal{F} = 1$ [see (2.9)], we may recognize the *cross-spectrum*,⁵

$$\hat{C}_{x_1 x_2}(f) = \overline{\hat{x}_1(f)} \hat{x}_2(f). \quad (2.18)$$

which is frequently encountered in signal processing. For $N = 3$, the correlation is

$$C_{x_1 x_2 x_3}(t_1, t_2) = \int_{-\infty}^{\infty} d\tau x_1(\tau) x_2(t_1 + \tau) x_3(t_2 + \tau). \quad (2.19)$$

Again converting the arguments of x_2 and x_3 with (2.8) leaves

$$\begin{aligned} C_{x_1 x_2 x_3}(t_1, t_2) &= \int_{-\infty}^{\infty} d\tau x_1(\tau) \int_{-\infty}^{\infty} df_1 \hat{x}_2(f_1) e^{2\pi i f_1(t_1+\tau)} \int_{-\infty}^{\infty} df_2 \hat{x}_3(f_2) e^{2\pi i f_2(t_2+\tau)}, \\ &= \int_{-\infty}^{\infty} df_1 \hat{x}_2(f_1) e^{2\pi i f_1 t_1} \int_{-\infty}^{\infty} df_2 \hat{x}_3(f_2) e^{2\pi i f_2 t_2} \int_{-\infty}^{\infty} d\tau x_1(\tau) e^{2\pi i(f_1+f_2)\tau}, \\ &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} df_1 df_2 \overline{\hat{x}_1(f_1+f_2)} \hat{x}_2(f_1) \hat{x}_3(f_2) e^{2\pi i(f_1 t_1 + f_2 t_2)}, \\ &= \mathcal{F}^{-1} \left\{ \overline{\hat{x}_1(f_1+f_2)} \hat{x}_2(f_1) \hat{x}_3(f_2) \right\} (t_1, t_2), \end{aligned} \quad (2.20)$$

where the last line follows by inspection of (2.13). Therefore, we conclude

$$\hat{C}_{x_1 x_2 x_3}(f_1, f_2) = \overline{\hat{x}_1(f_1+f_2)} \hat{x}_2(f_1) \hat{x}_3(f_2), \quad (2.21)$$

and may quickly derive (via induction) the Fourier transform of the N -correlation for all $N > 1$, known as the N -spectrum,

$$\hat{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}) = \overline{\hat{x}_1 \left(\sum_{j=1}^{N-1} f_j \right)} \prod_{k=1}^{N-1} \hat{x}_{k+1}(f_k) \in \mathbb{C}. \quad (2.22)$$

Now, consider the existence of a distribution of N -spectra, which might correspond to a change over time, say.⁶ Denoting the *expectation value* with angled brackets $\langle \cdot \rangle$, we have

$$\mathcal{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}) \equiv \left\langle \hat{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}) \right\rangle, \quad (2.23)$$

where we have defined the *cumulative N -spectrum* $\mathcal{C}_{x_1 \dots x_N}$. When $N = 3$, this quantity is known as the *bispectrum*,

$$\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) \equiv \left\langle \overline{\hat{x}_1(f_1+f_2)} \hat{x}_2(f_1) \hat{x}_3(f_2) \right\rangle, \quad (2.24)$$

which is the main object of study in this report.

⁵When $x_1 = x_2 = x$, we recover $\hat{C}_{xx}(f) = |\hat{x}(f)|^2$, known as the *power spectrum* of $x(t)$.

⁶Strictly speaking, the N -spectrum is not a function of time. However, we might imagine it characterized by a time-like parameter τ . For instance, let the functions y_k be defined $y_k(t, \tau) = x_k(t)w(t - \tau)$, where w is some compactly-supported weight function. Then, $\hat{C}_{y_1 \dots y_N} = \hat{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}, \tau)$. This will be discussed more in §3.2.

2.3 N -Coherence

Recalling the well-known Cauchy-Schwarz inequality,

$$\left| \sum_i \overline{u_i} v_i \right|^2 \leq \left(\sum_i |u_i|^2 \right) \left(\sum_i |v_i|^2 \right), \quad (2.25)$$

using $u = \hat{x}_1 \left(\sum_{j=1}^{N-1} f_j \right)$ and $v = \prod_{k=1}^{N-1} \hat{x}_{k+1}(f_k)$ provides a way to normalize the N -spectrum, such that it is real-valued and bound to the interval $[0, 1]$. Explicitly, (2.25) becomes

$$\left| \left\langle \overline{\hat{x}_1 \left(\sum_{j=1}^{N-1} f_j \right)} \prod_{k=1}^{N-1} \hat{x}_{k+1}(f_k) \right\rangle \right|^2 \leq \left\langle \left| \hat{x}_1 \left(\sum_{j=1}^{N-1} f_j \right) \right|^2 \right\rangle \left\langle \left| \prod_{k=1}^{N-1} \hat{x}_{k+1}(f_k) \right|^2 \right\rangle. \quad (2.26)$$

Noticing the L.H.S. is the squared modulus of the cumulative N -spectrum [compare with (2.22) & (2.23)], we define the *squared N -coherence spectrum* as

$$c_{x_1 \dots x_N}^2(f_1, \dots, f_{N-1}) = \frac{|\mathcal{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1})|^2}{\left\langle \left| \hat{x}_1 \left(\sum_{j=1}^{N-1} f_j \right) \right|^2 \right\rangle \left\langle \left| \prod_{k=1}^{N-1} \hat{x}_{k+1}(f_k) \right|^2 \right\rangle} \in [0, 1]. \quad (2.27)$$

To get some intuition on this quantity, consider $N = 2$,

$$c_{x_1 x_2}^2(f) = \frac{\left| \left\langle \overline{\hat{x}_1(f)} \hat{x}_2(f) \right\rangle \right|^2}{\left\langle |\hat{x}_1(f)|^2 \right\rangle \left\langle |\hat{x}_2(f)|^2 \right\rangle}, \quad (2.28)$$

known as the *cross-coherence* spectrum. Notice that if $x_1 = x_2 = x$, we have

$$c_{xx}^2(f) = \frac{\left| \left\langle |\hat{x}(f)|^2 \right\rangle \right|^2}{\left\langle |\hat{x}(f)|^2 \right\rangle \left\langle |\hat{x}(f)|^2 \right\rangle} = 1 \quad \forall f. \quad (2.29)$$

That is, the Fourier spectrum $\hat{x}(f)$ is completely *coherent* with itself. Conversely, when $N = 3$, we find the *squared bicoherence spectrum*,

$$b_{x_1 x_2 x_3}^2 = \frac{|\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2)|^2}{\left\langle |\hat{x}_1(f_1 + f_2)|^2 \right\rangle \left\langle |\hat{x}_2(f_1) \hat{x}_3(f_2)|^2 \right\rangle}, \quad (2.30)$$

using the bispectrum as defined in (2.24). A quick calculation will show that the *auto-bicoherence spectrum* $b_x^2 \equiv b_{xxx}^2$ is the simplest nontrivial N -coherence spectrum of a single function $x(t)$.

A note on conventions

$\mathcal{U}_p$ to now, we have been careful to define the N -correlation (2.14) to align with the well-established convention for cross-correlation (2.15), where the first function (x_1 , say) is conjugated upon Fourier transformation. However, for $N \geq 3$, there is no set rule, and we are free to circularly permute the indices of the functions x_k . Thus we will henceforth employ the notation

$$\boxed{\begin{aligned} \mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) &= \left\langle \hat{x}_1(f_1) \hat{x}_2(f_2) \overline{\hat{x}_3(f_1 + f_2)} \right\rangle \\ b_{x_1 x_2 x_3}^2(f_1, f_2) &= \frac{|\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2)|^2}{\left\langle |\hat{x}_1(f_1) \hat{x}_2(f_2)|^2 \right\rangle \left\langle |\hat{x}_3(f_1 + f_2)|^2 \right\rangle} \end{aligned}}, \quad (2.31)$$

which is somewhat more natural.

3 Implementation

3.1 Discretization

Of course, data taken in the lab is generally *digitized*, and so we must adjust our theory to properly encompass discrete signals — hence our focus on time-series analysis. For our purposes, a *time-series* is defined as a sequence of N real-valued data *sampled* at regularly spaced intervals in time, at a *sampling rate* f_s . That is, the time-series x is given by

$$x[n] \equiv \{x[t_n]\}_{n=0}^{N-1}, \quad (3.1)$$

where $t_n = n\Delta t$, using $\Delta t = 1/f_s$ as the *sampling period*. [Note that we use square brackets instead of parentheses to indicate the piece-wise nature of x .] Additionally, we continue to assume real-valued measurements ($x \in \mathbb{R}$). Now, the discrete Fourier transform⁷ (DFT) is analogous to the calculation of Fourier series coefficients (2.4),

$$\text{DFT}\{x\}[f] = \hat{x}[k] \equiv \{\hat{x}[f_k]\}_{k=0}^{N-1} = \frac{1}{N} \sum_{n=0}^{N-1} x[t_n] \exp[-2\pi i f_k t_n], \quad (3.2)$$

using $f_k = k/N\Delta t$ [see (2.6)]. For the time-series $x[n] = \exp[2\pi i f_0 t_n]$,⁸

$$\begin{aligned} \hat{x}[k] &= \frac{1}{N} \sum_{n=0}^{N-1} \exp[2\pi i f_0 n\Delta t] \exp\left[-\frac{2\pi i n k}{N}\right], \\ &= \frac{1}{N} \sum_{n=0}^{N-1} \exp\left[2\pi i n \left(f_0 \Delta t - \frac{k}{N}\right)\right]. \end{aligned} \quad (3.3)$$

Recognizing this is a geometric sum, we use

$$\sum_{n=0}^{N-1} a^n = 1 + a + a^2 + \dots + a^{N-1} = \frac{1 - a^N}{1 - a}, \quad (3.4)$$

which implies

$$\hat{x}[k] = \frac{1}{N} \frac{1 - \exp\left[2\pi i N \left(f_0 \Delta t - \frac{k}{N}\right)\right]}{1 - \exp\left[2\pi i \left(f_0 \Delta t - \frac{k}{N}\right)\right]}. \quad (3.5)$$

Further, notice that

$$\left| \frac{1 - e^{iNu}}{1 - e^{iu}} \right|^2 = \frac{2(1 - \cos(Nu))}{2(1 - \cos u)} = \frac{\sin^2(Nu/2)}{\sin^2(u/2)}, \quad (3.6)$$

from which we conclude the power spectrum $|\hat{x}|^2$ for $x[n] = \exp[2\pi i f_0 t_n]$ is given by

$$|\hat{x}[k]|^2 = \frac{1}{N^2} \frac{\sin^2\left[\pi N \left(f_0 \Delta t - \frac{k}{N}\right)\right]}{\sin^2\left[\pi \left(f_0 \Delta t - \frac{k}{N}\right)\right]}. \quad (3.7)$$

Finally, consider the exponential in (3.5) when $k = N - k'$ and $f_0 = -|f_0|$,

$$\exp\left[2\pi i \left(f_0 \Delta t - \frac{(N - k')}{N}\right)\right] = \exp\left[-2\pi i \left(|f_0| \Delta t - \frac{k'}{N}\right)\right] \exp[-2\pi i] \overset{1}{\rightarrow} \quad (3.8)$$

⁷It is important to note that many DFT implementations (`fft` in Matlab, say) do not include the $1/N$ prefactor in (3.2).

⁸You might notice that $x \in \mathbb{C}$, which violates our assumption on real-valued measurements. However, because real-valued time-series may be built from $2\text{Re}(x) = x + \bar{x}$, it will not affect our conclusions about $x \in \mathbb{R}$.

When the record length of the time-series $T = N\Delta t$ is an integral multiple of the oscillation period, we have $f_0 N\Delta t = f_0 T \equiv p \in \mathbb{Z}$, and the numerator of (3.5) vanishes by virtue of $\exp[2\pi i(p - k)] = 1$. Furthermore, if $|p| < N$, we may use (3.3) & (3.8) to show there exists a unique k_0 satisfying

$$\hat{x}[k] = \delta_{kk_0}, \quad k_0 = \begin{cases} f_0 N\Delta t, & f_0 \geq 0, \\ N - |f_0| N\Delta t, & f_0 < 0. \end{cases} \quad (3.9)$$

In this special case, we acknowledge the clear analogy with Fourier series. But what if $f_0 T \notin \mathbb{Z}$? Then, the discrete orthogonality condition (3.9) does *not* hold, and the full result (3.5) must be used — this is called *spectral leakage* (see Fig. 1). Naturally, it is unlikely we know *a priori* if $f_0 N\Delta t$ is an integer, and thus leakage is all but impossible to avoid. [We deal with the implications of this fact in the following section.] Now, let our time-series be given by $x[n] = (1/2)(\exp[2\pi i f_0 t_n] + \exp[-2\pi i f_0 t_n]) = \cos[2\pi f_0 t_n]$. From (3.3) & (3.8), we see that the DFT will contain both positive and negative frequency components, which implies $\hat{x}[k] = \hat{x}[N - k]$. Indeed, the *Nyquist-Shannon sampling theorem* dictates that for real-valued signals, frequencies above the *Nyquist frequency* $f_{\text{Nyq}} \equiv f_s/2$ will be *aliased*, the technical term for the expression of this symmetry. Thus, $f \leq f_{\text{Nyq}}$, leaving

$$\frac{k}{N\Delta t} \leq \frac{1}{2\Delta t} \implies \boxed{k_{\text{Nyq}} = \lfloor N/2 \rfloor}, \quad (3.10)$$

where $\lfloor \cdot \rfloor$ indicates the *floor function*. This symmetry is clearly visible in Figure 1.

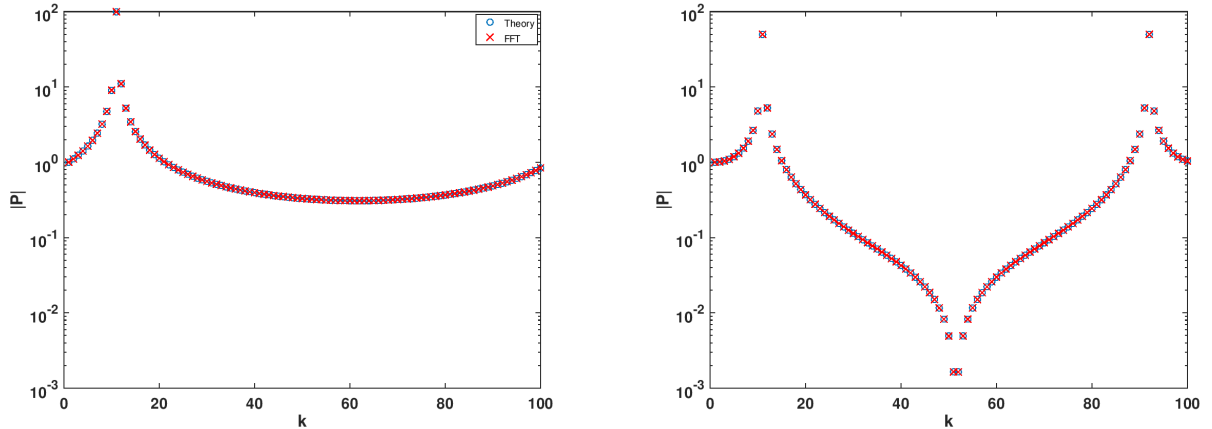


Figure 1: (LEFT) Comparison of theoretical result $N|\hat{x}[k]|$ as given by (3.7) [blue circles] and Matlab's inbuilt `fft` function [red $\times$ s]; note logarithmic scale. Relevant parameters are $f_0 = 10$ Hz, $\Delta t = 10$ ms, and $N = 101$. If $N \rightarrow 100$, both methods provide a single nonzero value at $k = 10$. (RIGHT) Same analysis, for cosine time-series, which prominently displays aliasing.

Consequently, the discrete bispectrum and discrete squared bicoherence spectrum are then, respectively,

$$\left. \begin{aligned} \mathcal{B}_{x_1 x_2 x_3}[j, k] &= \langle \hat{x}_1[j] \hat{x}_2[k] \hat{x}_3[j+k] \rangle, \\ b_{x_1 x_2 x_3}^2[j, k] &= \frac{|\mathcal{B}_{x_1 x_2 x_3}[j, k]|^2}{\langle |\hat{x}_1[j] \hat{x}_2[k]|^2 \rangle \langle |\hat{x}_3[j+k]|^2 \rangle + \varepsilon}, \end{aligned} \right| \quad j+k \leq \frac{N}{2}, \quad (3.11)$$

where $\varepsilon \ll 1$ is a small number used to avoid evaluation of $0/0 = \text{NaN}$. Though we have not formally shown that our results for the N -spectrum carry over unchanged to the discrete case, it is straightforward to do so. To avoid beleaguering the point, we forgo the explicit calculation and direct the reader to the reasoning of (3.16).

3.2 Windowing

Suppression of spectral information leakage is paramount to a viable analysis. To reduce the influence of these edge effects, we apply some *window* to our data, weighting the time-series so that the samples at $x[n = 0]$ or $x[n = N - 1]$ are negligible when compared to those near $x[n = N/2]$. Let us, for simplicity, briefly consider again the continuous function $x(t)$. Though we have not explicitly derived it, it is an immediate consequence of (2.17) that the Fourier transform of a product is a *convolution*⁹ in frequency space. That is,

$$\mathcal{F}\{x(t)w(t)\}(f) = \int_{-\infty}^{\infty} \hat{x}(\zeta)\hat{w}(f - \zeta)d\zeta = (\hat{x} * \hat{w})(f), \quad (3.12)$$

where w and $\hat{w}$ are the window and its transform, respectively. Hence, we may deduce the effect of a particular window by simply analyzing its Fourier spectrum. Since we know, for example, that $\mathcal{F}\{1\} = \delta(f)$ [see (2.10)], we might immediately recognize $(\hat{x} * \delta)(f) = \hat{x}(f)$, which says that the constant window $w(t) = 1$ yields the original Fourier spectrum. Also, in the idealized case where $x(t) = e^{2\pi i f_0 t} \iff \hat{x}(f) = \delta(f - f_0)$, any integrable window leads to

$$\mathcal{F}\{e^{2\pi i f_0 t} w(t)\} = \int_{-\infty}^{\infty} \delta(\zeta - f_0)\hat{w}(\zeta)d\zeta = \hat{w}(f - f_0), \quad (3.13)$$

which is simply the window's own transform, but peaked at $f = f_0$. Clearly, we want to acknowledge more interesting cases. For instance, imagine that we use this idea of windowing to gain some time-resolution (as discussed in Footnote 6), via the parametrized function

$$y(t, \tau) = x(t, \tau)w(t - \tau). \quad (3.14)$$

From (2.7), we see that a shift in time yields a phase factor in the Fourier domain, $\mathcal{F}\{x(t - \tau)\}(f) = e^{-2\pi i f \tau} \hat{x}(f)$. Using the contrived function $x(t, \tau) = e^{-2\pi i f_0(\tau)t}$, we find that (3.14) becomes, after a Fourier transformation,

$$\hat{y}(f, \tau) = \hat{w}(f - f_0(\tau)) e^{-2\pi i f \tau}. \quad (3.15)$$

The power spectrum $|\hat{y}|^2$ is clearly $|\hat{w}(f - f_0(\tau))|^2$, which helps us understand (in a schematic sense) how we might formally interpret time-dependence in frequency space. Unfortunately, things are not always as simple (read: clean) in the discretized case! But, before we get ahead of ourselves, we should show that the convolution theorem (3.12) holds for the DFT. Being a bit loose with our notation, we find

$$\begin{aligned} (\hat{x} * \hat{w})[k] &= \sum_{j=0}^{N-1} \hat{x}[j]\hat{w}[k - j] \\ &= \sum_{j=0}^{N-1} \hat{x}[j] \left(\frac{1}{N} \sum_{n=0}^{N-1} w[n] \exp \left[-\frac{2\pi i n(k - j)}{N} \right] \right) \\ &= \frac{1}{N} \sum_{n=0}^{N-1} w[n] \exp \left[-\frac{2\pi i n k}{N} \right] \left(\sum_{j=0}^{N-1} \hat{x}[j] \exp \left[\frac{2\pi i n j}{N} \right] \right) \\ &= \frac{1}{N} \sum_{n=0}^{N-1} x[n]w[n] \exp \left[-\frac{2\pi i n k}{N} \right], \end{aligned} \quad (3.16)$$

where in the second line¹⁰ we substituted with (3.2), and in the last line we have used the *inverse* DFT,

$$x[n] = \sum_{k=0}^{N-1} \hat{x}[k] \exp \left[\frac{2\pi i n k}{N} \right], \quad (3.17)$$

⁹The convolution of two functions $x * y$ is equivalent to the cross-correlation of $x(-t)$ and $y(t)$. In our notation, $(x * y)(t) = C_{xy}(t)$, where $\underline{x}(t) \equiv x(-t)$.

¹⁰You may be rightly distressed by the cavalier use of the term $\hat{w}[k - j]$, which seems to involve negative indices. This is a well-founded criticism, and is addressed by noticing that the DFT is formally periodic $\hat{x}[k + N] = \hat{x}[k]$. More technically, we are employing *circular convolution*.

which may be derived from (3.2) & (3.9). Thus,

$$\text{DFT}\{x[n]w[n]\}[k] = \sum_{j=0}^{N-1} \hat{x}[j]\hat{w}[k-j], \quad (3.18)$$

analogously to (3.12). Despite this, using $x[n] = \exp[2\pi i f_0 n \Delta t]$ and the general relation (3.5), we find

$$\text{DFT}\{x[n]w[n]\}[k] = \frac{1}{N} \sum_{j=0}^{N-1} \left(\frac{1 - \exp[2\pi i N (f_0 \Delta t - \frac{j}{N})]}{1 - \exp[2\pi i (f_0 \Delta t - \frac{j}{N})]} \right) \hat{w}[k-j], \quad (3.19)$$

which certainly does not have the convenient form of (3.13). To avoid the tedium of working through such sums, we present the resulting transform for various powers of the *half-cycle sine*,

$$w[n] = \sin^\alpha \left[\frac{\pi n}{N-1} \right]. \quad (3.20)$$

Certain values of the exponent α possess an accepted nomenclature: $\alpha = 0$ is (naturally) the *rectangular window*, $\alpha = 1$ provides the *sine window*, and $\alpha = 2$ is the *Hann window*. As can be seen in Figure 2, the effective width of the window (in time) is related to its localization in frequency space. Figure 3 shows the effect of a series of windows on the real-valued time-series $x[n] = \cos[2\pi f_0 n \Delta t]$. Notice that as the window narrows, spectral leakage begins to diminish (spectral content is greatly reduced away from the peaks), but the width of the peaks increase in frequency space.

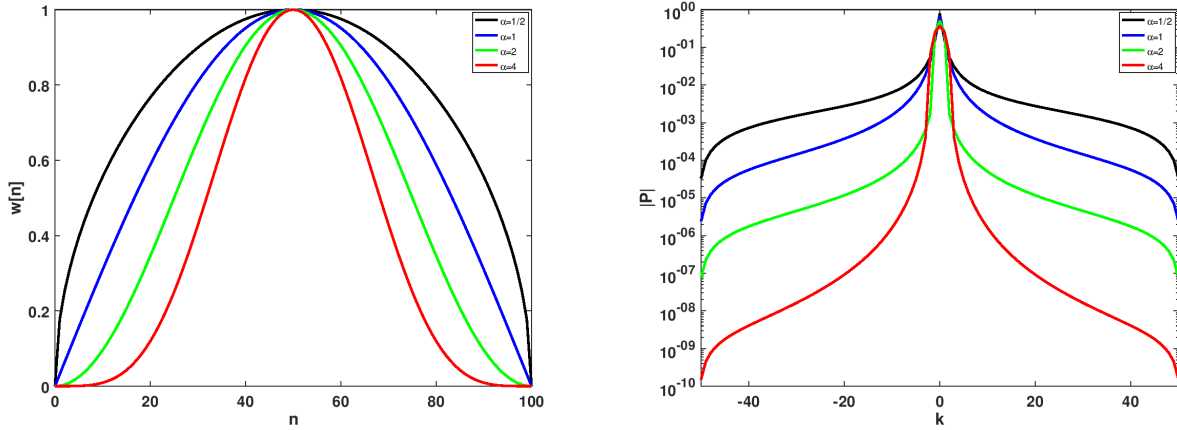


Figure 2: (LEFT) Visualization of various powers of the half-cycle sine (3.20) for $N = 101$, with $\alpha = 1/2$ [black], $\alpha = 1$ [blue], $\alpha = 2$ [green], $\alpha = 4$ [red]. (RIGHT) Absolute value of the corresponding Fourier transforms (DFTs), shifted such that symmetry about $k = 0$ is manifest.

This is directly related to an uncertainty principle, which we quickly derive. First, notice *Parseval's theorem*¹¹ (for $x \in \mathbb{R}$),

$$\begin{aligned} \int_{-\infty}^{\infty} |\hat{x}(f)|^2 df &= \int_{-\infty}^{\infty} df \int_{-\infty}^{\infty} x(t) e^{2\pi i f t} dt \int_{-\infty}^{\infty} x(t') e^{-2\pi i f t'} dt', \\ &= \int_{-\infty}^{\infty} x(t) dt \int_{-\infty}^{\infty} x(t') dt' \int_{-\infty}^{\infty} df e^{2\pi i f (t-t')}, \\ &= \int_{-\infty}^{\infty} x^2(t) dt, \end{aligned} \quad (3.21)$$

¹¹For discrete analyses, this becomes $N \sum_k |\hat{x}[k]|^2 = \sum_n x^2[n]$.

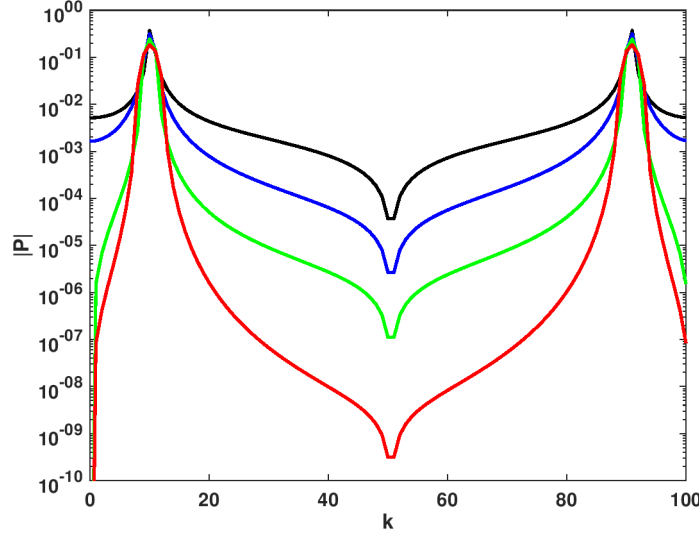


Figure 3: Absolute value of DFT coefficients for $x[n] = \cos[2\pi f_0 n \Delta t]$, when windowed with the half-cycle sine (3.20) for $N = 101$, with $\alpha = 1/2$ [black], $\alpha = 1$ [blue], $\alpha = 2$ [green], $\alpha = 4$ [red]. See Fig. 1 for relevant parameters, and to compare with previous Fourier transform results. Notice the reduction in spectral content away from the (widening) peaks as the window narrows.

where we made use of (2.10) in the first line and (2.7) in the second. Now, taking the derivative with respect to time of (2.8), we deduce

$$x'(t) = \int_{-\infty}^{\infty} 2\pi i f \hat{x}(f) e^{2\pi i f t} df \iff \mathcal{F}\{x'(t)\} = 2\pi i f \hat{x}(f). \quad (3.22)$$

With these facts in hand, we see that integration by parts yields

$$\int_{-\infty}^{\infty} x^2(t) dt = [tx^2(t)]_{-\infty}^{\infty} - 2 \int_{-\infty}^{\infty} tx(t)x'(t) dt. \quad (3.23)$$

If we demand the function $x(t)$ satisfies $\lim_{|t| \rightarrow \infty} tx^2(t) = 0$, the Cauchy-Schwartz inequality (2.25) implies

$$\begin{aligned} \left(\int_{-\infty}^{\infty} x^2(t) dt \right)^2 &= 4 \left(\int_{-\infty}^{\infty} tx(t)x'(t) dt \right)^2 \leq 4 \left(\int_{-\infty}^{\infty} t^2 x^2(t) dt \right) \left(\int_{-\infty}^{\infty} [x'(t)]^2 dt \right) \\ &\leq 4 \left(\int_{-\infty}^{\infty} t^2 x^2(t) dt \right) \left(\int_{-\infty}^{\infty} 4\pi^2 f^2 |\hat{x}(f)|^2 df \right), \end{aligned} \quad (3.24)$$

where in moving to the second line we used Parseval's theorem (3.21) with (3.22). Pulling the scalars outside the rightmost integral, and defining the *variances*

$$\sigma_t^2 = \int_{-\infty}^{\infty} t^2 x^2(t) dt, \quad \text{and} \quad \sigma_f^2 = \int_{-\infty}^{\infty} f^2 |\hat{x}(f)|^2 df, \quad (3.25)$$

we find that a function and its transform must satisfy¹² the *Heisenberg-Gabor limit*,

$$\frac{\sigma_t \sigma_f}{||x||^2} \geq \frac{1}{4\pi}. \quad (3.26)$$

Similarly to quantum mechanics, equality is met with (you can check!) a Gaussian $x(u) = \hat{x}(u) = e^{-\pi u^2}$.

¹²Here, we use the so-called L^2 norm $||x|| \equiv (\int_{\mathbb{R}} |x|^2 dt)^{1/2}$.

3.3 Bi-frequency space

As an example, we examine the simplest possible circumstance in bispectral analysis: given $\{x_k\}_{k=1}^3$ such that

$$x_k(t) = \cos(2\pi\nu_k t + \varphi_k), \quad (3.27)$$

where ν_k and φ_k are arbitrary frequencies and phases, respectively. It is clear from (2.11) that

$$\hat{x}_k(f) = \frac{e^{i\varphi_k}}{2} [\delta(f - \nu_k) + \delta(f + \nu_k)], \quad (3.28)$$

and hence

$$\begin{aligned} \mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) &= \hat{x}_1(f_1) \hat{x}_2(f_2) \overline{\hat{x}_3(f_1 + f_2)} \\ &= e^{i(\varphi_1 + \varphi_2 - \varphi_3)} [\delta(f_1 + f_2 - \nu_3) + \delta(f_1 + f_2 + \nu_3)] \\ &\quad \times [\delta(f_1 - \nu_1) + \delta(f_1 + \nu_1)] [\delta(f_2 - \nu_2) + \delta(f_2 + \nu_2)]. \end{aligned} \quad (3.29)$$

For this to be nonzero, the frequencies ν_k must satisfy at least one of the equations $\pm\nu_1 \pm \nu_2 \mp \nu_3 = 0$, where the signs may be chosen independently (*i.e.*, there are 8 formal solutions). Figure 4 provides a graphical argument which shows that the ambiguity of signs ($\nu_k \in \mathbb{R}$) leaves only two simultaneous solutions. These are manifested in the linearly dependent equations,

$$\begin{aligned} \nu_1 + \nu_2 &= \nu_3, \\ -\nu_1 - \nu_2 &= -\nu_3. \end{aligned} \quad (3.30)$$

However, because the arrangement of ν_k is arbitrary, we actually have $3!$ times as many solutions. That is, given $(f_1, f_2) = (\nu_1, \nu_2)$ satisfying (3.30), the 12 permutations of $\{x_1, x_2, x_3\}$ in $\mathcal{B}_{x_1 x_2 x_3}$ provide other nonzero points in the bispectrum, and are equivalent to rearranging the above relations [see Fig. 4].

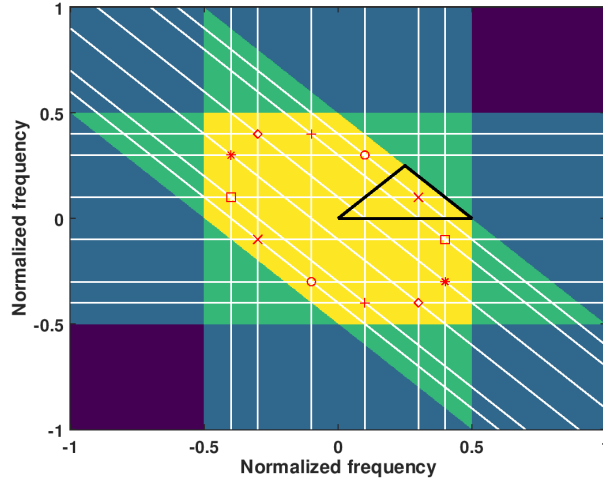


Figure 4: Visualization of bi-frequency space with axes normalized to the sampling frequency. Red symbols highlight the 12 ordered pairs $(0.1, 0.3)$, $(0.3, 0.1)$, $(0.4, -0.1)$, $(0.4, -0.3)$, $(0.3, -0.4)$, $(0.1, -0.4)$, $(-0.1, -0.3)$, $(-0.3, -0.1)$, $(-0.4, 0.3)$, $(-0.3, 0.4)$, $(-0.1, 0.4)$ with matching symbols indicating conjugate pairs. White lines are meant to aid in recognition of crossings, and color is related to set intersections. Yellow area shows domain of validity as given by (3.31), and triangle outlines sufficient region for auto-bispectral analyses.

Furthermore, any choice of ν_k which satisfies (3.30) will correspond to a conjugate pair in bi-frequency space. When considering digitized data, the discrete bispectrum exists on the domain

$$\mathcal{D} = \{f_1 \leq f_{\text{Nyq}}\} \cap \{f_2 \leq f_{\text{Nyq}}\} \cap \{f_1 + f_2 \leq f_{\text{Nyq}}\}, \quad (3.31)$$

as demarcated by the yellow region of Figure 4. In our example of cross-bicoherence $\mathcal{B}_{x_1 x_2 x_3}$, only two of these solutions may be satisfied simultaneously; however, if we calculate the auto-bicoherence $\mathcal{B}_{xxx}$ of the functions' sum (*i.e.*, if $x = x_1 + x_2 + x_3$), then all 12 points will be populated in bi-frequency space, as in Figure 5. Due to this symmetry, the auto-bispectrum need only be calculated for a reduced domain [see triangle in Figure 4]. This will be discussed more in §4.2.

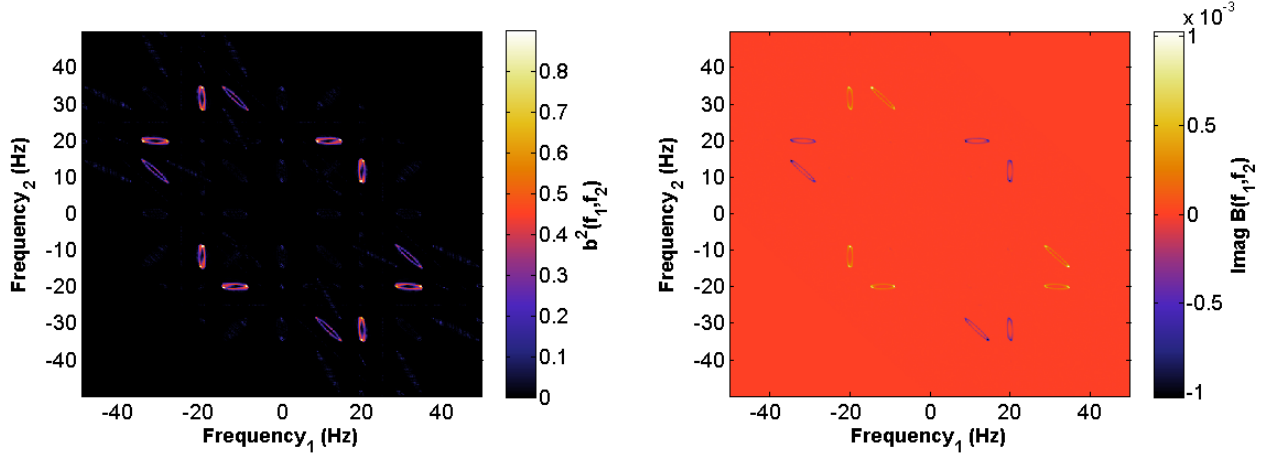


Figure 5: (LEFT) Bicoherence spectrum and (RIGHT) imaginary part of auto-bispectrum for time-series containing 3 frequency-modulated sinusoids which are in constant satisfaction of the sum-frequency condition. The right-hand plot displays the conjugacy relationships between points.

3.4 Phase dynamics

Before we tackle calculating the bispectrum (2.24) or bicoherence spectrum (2.30), we should take a moment to appreciate *why* it is relevant. Going all the way back to §2.2, consider rewriting the N -spectrum (2.22) in the following way,

$$\hat{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}) = \left| \hat{C}_{x_1 \dots x_N}(f_1, \dots, f_{N-1}) \right| e^{i\varsigma(f_1, \dots, f_{N-1})}. \quad (3.32)$$

We call the quantity $\varsigma(f_1, \dots, f_{N-1})$ the N -*phase*. Writing the Fourier components in a similar fashion,

$$\hat{x}_k(f) = |\hat{x}_k(f)| e^{i\phi_k(f)}, \quad (3.33)$$

we see by inspection of (2.22) and (3.32) that

$$\varsigma(f_1, \dots, f_{N-1}) = \left(\sum_{k=1}^{N-1} \phi_k(f_k) \right) - \phi_N \left(\sum_{j=1}^{N-1} f_j \right), \quad (3.34)$$

where we have employed the convention as described in §2.3. For $N = 3$, we have the *biphase* β ,

$$\beta(f_1, f_2) = \phi_1(f_1) + \phi_2(f_2) - \phi_3(f_1 + f_2). \quad (3.35)$$

What we wish to understand is the way in which this phase affects the bispectrum (or more generally, the cumulative N -spectrum (2.23)). First, let's quietly clear up a point we glossed over in §2.2 & §2.3, which is the nature of the expectation value. For instance, a relatively general definition would be

$$\langle u \rangle = \int_{-\infty}^{\infty} \rho(\tau) u(\tau) d\tau, \quad (3.36)$$

where u is some arbitrary function and ρ is a *probability density*, which satisfies $\int_{-\infty}^{\infty} \rho d\tau = 1$. However, many times we will simply assume ρ is constant and nonzero only on some interval $\tau \in [\tau_i, \tau_f]$. That is,

$$\langle u \rangle = \frac{1}{\tau_f - \tau_i} \int_{\tau_i}^{\tau_f} u(\tau) d\tau. \quad (3.37)$$

In this case, consider the bispectral dynamics of the functions $x_k(t, \tau) = \cos(2\pi\nu_k t + \varphi_k(\tau))$, where we contrive the phases to have a formal dependence on τ . Notice that when Fourier amplitudes are not a function of τ , the bispectrum is proportional to the average of a phasor, via

$$\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) \propto \left\langle e^{i\beta(f_1, f_2, \tau)} \right\rangle = \frac{1}{\tau_f - \tau_i} \int_{\tau_i}^{\tau_f} e^{i\beta(f_1, f_2, \tau)} d\tau, \quad (3.38)$$

where the biphas $\beta(f_1, f_2, \tau)$ is now parametrized by τ [see the prefactor in (3.29)]. For generality, let

$$\beta(\tau) = \beta_0 + a\tau + z \sin(b\tau), \quad (3.39)$$

where β_0, z are dimensionless constants, a, b are constants with units of frequency, and we have suppressed the f_1, f_2 dependence for brevity. Defining $\tau_i = 0$ and $b\tau_f = 2\pi$, we find

$$\begin{aligned} \mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) &\propto \frac{b}{2\pi} \int_0^{2\pi/b} e^{i\beta_0} e^{ia\tau} e^{iz \sin(b\tau)} d\tau, \\ &\propto \frac{e^{i\beta_0}}{2\pi} \int_0^{2\pi} e^{iau/b} e^{iz \sin u} du, \\ &\propto -\frac{e^{i\beta_0}}{2\pi} \int_{\pi}^{-\pi} e^{ia\pi/b} e^{-iav/b} e^{iz \sin v} dv, \\ &\propto \frac{e^{i(\beta_0 - a\pi/b)}}{2\pi} \int_{-\pi}^{\pi} e^{-iav/b} e^{iz \sin v} dv, \end{aligned} \quad (3.40)$$

where we have made the changes of variable $u \rightarrow b\tau$ and $v \rightarrow \pi - u$ in the second and third line, respectively. For context, recall *Bessel's integral*,

$$J_n(z) = \frac{1}{2\pi} \int_{-\pi}^{\pi} e^{iz \sin u} e^{-inu} du, \quad (3.41)$$

where $J_n(z)$ is a Bessel function of the first kind, of order n . Note that this is simply the coefficient c_n in the Fourier series expansion of $x(t) = e^{iz \sin t}$ [see (2.5) with $T = 2\pi$]. In the most convenient case, we assume $a/b \in \mathbb{Z}$, though it is not difficult to generalize.¹³ Thus we see by inspection that

$$\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) \propto e^{i(\beta_0 - a\pi/b)} J_{a/b}(z). \quad (3.42)$$

A kernel of salience here is that for $z = 0$, Bessel functions satisfy $J_n(0) = \delta_{n0}$. Hence, the integral will tend to vanish if the biphas is a linear function $\beta(\tau) = \beta_0 + a\tau$, except when $a \rightarrow 0$. Otherwise, an oscillatory biphas ($z \neq 0$) does not generally lead to a vanishing bispectrum. What does this tell us? That only certain relationships between the phases of Fourier coefficients at frequencies f_1, f_2 , and $f_3 = f_1 + f_2$ lead to nonzero bispectral features at (f_1, f_2) . Specifically, linear (or random) relationships are not *phase-coherent*. Conversely, consider a quadratic nonlinearity,

$$x_3(t) = \cos(\omega_1 t + \varphi_1) \cos(\omega_2 t + \varphi_2) = \frac{1}{2} (\cos[(\omega_1 + \omega_2)t + (\varphi_1 + \varphi_2)] + \cos[(\omega_1 - \omega_2)t + (\varphi_1 - \varphi_2)]), \quad (3.43)$$

where $\omega_k = 2\pi\nu_k$. Repeating the analysis of §3.3 implies that $\beta(\nu_1, \nu_2) = \varphi_1 + \varphi_2 - (\varphi_1 + \varphi_2) = 0$, or that quadratic nonlinearities *always* exhibit a stationary biphas (they are completely phase-coherent).

¹³Bessel functions of nonintegral orders may be deduced with the help of contour integration, and are beyond the scope of this document.

3.5 Algorithm

When actually implementing a bispectral analysis on discrete data, we will follow a basic strategy:

$$\boxed{\text{Perform DFT}} \implies \boxed{\text{Calculate } \mathcal{B}} \implies \boxed{\text{Calculate } \mathbf{b}^2}$$

In practice, our data will encompass many ($N > 10^3$) samples over some interval of time, and thus time-resolution may be important. As alluded to in §3.2, a natural way to accomplish this is by using a window whose peak is controlled by a time-like parameter. The so-called *Welch method*, which consists of translating a *subinterval* of a given size s (in samples) by a set number of samples λ (the *step*), provides a robust way to implement this. Given the time-series $x[n]$, we form subintervals $x[n]_m$ via

$$x[n]_m \equiv x[n + m\lambda]w[n], \quad n \in [0, s - 1], \quad (3.44)$$

for $m \in \mathbb{N}$, and $w[n]$ is some window function of width s samples. Crucially, m must satisfy

$$s + m\lambda - 1 \leq N - 1, \quad (3.45)$$

which implies $m \leq \lfloor (N - s)/\lambda \rfloor$. A more visual demonstration is given in Figure 6.

$$\begin{aligned} & \underbrace{\{x[t_0], x[t_1], x[t_2], \dots, x[t_{s-1}]\}}_{m=0}, x[t_s], x[t_{s+1}], \dots, x[t_{N-1}] \\ & \{x[t_0], x[t_1], x[t_2], \dots, x[t_{s-1}], \underbrace{x[t_s], x[t_{s+1}], \dots, x[t_{N-1}]}_{m=1}\} \\ & \{x[t_0], x[t_1], \underbrace{x[t_2], \dots, x[t_{s-1}], x[t_s], x[t_{s+1}], \dots, x[t_{N-1}]}_{m=2}\} \\ & \vdots \\ & \{x[t_0], x[t_1], \dots, x[t_{N-s-1}], \underbrace{x[t_{N-s}], \dots, x[t_{N-2}], x[t_{N-1}]}_{m=\lfloor (N-s)/\lambda \rfloor - 1}\} \\ & \{x[t_0], x[t_1], \dots, x[t_{N-s-1}], \underbrace{x[t_{N-s}], \dots, x[t_{N-2}], x[t_{N-1}]}_{m=\lfloor (N-s)/\lambda \rfloor}\} \end{aligned}$$

Figure 6: Visualization of moving subinterval as used in Welch method for calculating power spectral density (PSD). Here, the subinterval has length s samples, with step size $\lambda = 1$ and unity window.

This approach provides a clear means to calculate the *spectrogram* as a concatenation of the power spectra,¹⁴

$$\text{spectrogram}\{x[n]\}[m, k] = |\hat{x}[k]_m|^2, \quad (3.46)$$

and the *power spectral density* as the square of the Fourier amplitudes' ensemble average,

$$\text{PSD}\{x[n]\}[k] = \langle |\hat{x}[k]_m|^2 \rangle = \left(\frac{1}{M} \sum_{m=0}^M |\hat{x}[k]_m| \right)^2, \quad (3.47)$$

where $M = \lfloor (N - s)/\lambda \rfloor$. The bispectrum and squared bicoherence spectrum are then, respectively

$$\boxed{\begin{aligned} \mathcal{B}_{x_1 x_2 x_3}[j, k] &= \frac{1}{M} \sum_{m=0}^M \hat{x}_1[j]_m \hat{x}_2[k]_m \overline{\hat{x}_3[j+k]_m}, \\ b_{x_1 x_2 x_3}^2[j, k] &= \frac{M^2 |\mathcal{B}_{x_1 x_2 x_3}[j, k]|^2}{\left(\sum_{m=0}^M |\hat{x}_1[j]_m \hat{x}_2[k]_m|^2 \right) \left(\sum_{m=0}^M |\hat{x}_3[j+k]_m|^2 \right) + \varepsilon}, \end{aligned} \quad \left| \quad j+k \leq \lfloor N/2 \rfloor \right.} \quad (3.48)$$

¹⁴For completeness, we have defined $\hat{x}[k]_m \equiv \text{DFT}\{x[n]_m\}$.

3.6 Wavelet analysis*

Zooming out a touch, we might consider the breadth of our journey before moving on. Stated roughly, one could claim the pinnacle of Fourier theory is contained in an expression similar to (3.14),

$$\mathcal{F}\{x(t)w(t-\tau)\}(f, \tau) = \int_{-\infty}^{\infty} dt x(t)w(t-\tau)e^{-2\pi ift}, \quad (3.49)$$

which offers both time and frequency resolution. [Indeed, this is the continuous analog of the spectrogram!] Setting aside the effect of τ (and questions of what a “time-like parameter” actually *means*), we notice that, abstractly, we are performing the operation

$$\text{“Information”} = \int_{-\infty}^{\infty} dt [\text{“Signal”} \times \text{“Thing”}]. \quad (3.50)$$

This feels reductive, but hear me out! Consider the generic functional,

$$\mathcal{Q}\{x; \psi\}(\alpha, \tau, \sigma) = \int_{-\infty}^{\infty} dt x(t)\psi(\alpha, t, \tau, \sigma), \quad (3.51)$$

defining $\mathcal{Q}(\alpha, \tau, \sigma)$ as a “transform” and $\psi(\alpha, t, \tau, \sigma)$ as an integrable weight function (we will be more specific in a moment). Notice that we have the special cases

$$\mathcal{Q}\{x; \psi\}(\alpha, \tau, \sigma) = \begin{cases} x(\tau) \iff \psi = \delta(t - \tau), \\ \int_{-\sigma}^{\sigma} x(t)dt \iff \psi = H(t + \sigma) - H(t - \sigma) \\ \hat{x}(\alpha) \iff \psi = e^{-2\pi i \alpha t}. \end{cases} \quad (3.52)$$

Hence, this general form recovers both the original signal, the Fourier transform, and anything in-between by judicious choice of ψ . Indeed, we’re being a bit *too* general! What would be perfect is an adjustable transform which, on one hand, retains the Fourier-mode intuition we’ve developed, and on the other, may highlight time or frequency at our whims. In particular, we use the unitless parameter α and the time-like parameter τ to scale and shift t ,

$$\mathcal{W}\{x; \Psi_{\sigma}\}(\alpha, \tau) = \int_{-\infty}^{\infty} dt x(t)\Psi_{\sigma}(\alpha(t - \tau)), \quad (3.53)$$

which we will call the *continuous wavelet transform* (CWT). The complex function $\Psi_{\sigma}(t)$ is known as a “wavelet” because (among other conditions) it is normalized with respect to the L^2 norm:¹⁵

$$\int_{-\infty}^{\infty} |\Psi_{\sigma}(\alpha t)|^2 dt = 1. \quad (3.54)$$

A bit of manipulation¹⁶ reveals that

$$\begin{aligned} \mathcal{W}\{x; \Psi_{\sigma}\}(\alpha, \tau) &= \int_{-\infty}^{\infty} dt x(t)\Psi_{\sigma}^*(\alpha(t - \tau)), \\ &= \int_{-\infty}^{\infty} dt x(t) \int_{-\infty}^{\infty} df \hat{\Psi}_{\sigma}^*(f) e^{-2\pi i \alpha f(t - \tau)}, \\ &= \int_{-\infty}^{\infty} df \hat{\Psi}_{\sigma}^*(f) e^{2\pi i \alpha f \tau} \int_{-\infty}^{\infty} dt x(t) e^{-2\pi i \alpha f t}, \\ &= \int_{-\infty}^{\infty} df \hat{x}(\alpha f) \hat{\Psi}_{\sigma}^*(f) e^{2\pi i \alpha f \tau}, \\ &= \frac{1}{\alpha} \mathcal{F}^{-1} \left\{ \hat{x}(f) \hat{\Psi}_{\sigma}^*(f/\alpha) \right\}(\tau), \end{aligned} \quad (3.55)$$

¹⁵I admit this might not be the most intuitive argument for the “wavelet” nomenclature, but consider instead a wave: $e^{2\pi i f t}$. The integral of the squared modulus is $\int_{-\infty}^{\infty} dt$, and is clearly not normalizable.

¹⁶For completeness, in moving to the second line we use the general relation $x^*(t) = \int_{-\infty}^{\infty} df \hat{x}^*(f) e^{-2\pi i f t}$.

which shows the CWT may be efficiently calculated with an inverse Fourier transform. To connect with our well-developed instincts for Fourier analysis, we choose the *Morlet* wavelet,

$$\Psi_\sigma(\alpha t) = A e^{-2\pi i \alpha f_0 t} \exp \left[-\frac{1}{2} \left(\frac{\alpha t}{\sigma} \right)^2 \right], \quad (3.56)$$

which is simply a complex exponential windowed by a Gaussian envelope. Hence, using the Morlet wavelet, we have for unity scale ($\alpha = 1$),

$$\widehat{\Psi}_\sigma(f) = \mathcal{F} \left\{ A e^{-2\pi i f_0 t} \exp \left[-\frac{t^2}{2\sigma^2} \right] \right\} = \mathcal{F} \{ A e^{-2\pi i f_0 t} \} * \mathcal{F} \left\{ \exp \left[-\frac{t^2}{2\sigma^2} \right] \right\}, \quad (3.57)$$

by the convolution theorem (3.12). The first transform (in the convolution) is trivial [*i.e.*, $A\delta(f - f_0)$; see (2.10)], so we need only find the transform of the Gaussian part [see (3.13)]. Explicitly, we wish to evaluate

$$\int_{-\infty}^{\infty} dt \exp \left[-\frac{t^2}{2\sigma^2} - 2\pi i f_0 t \right]. \quad (3.58)$$

As the integrand is of the form $\exp(-at^2 - bt)$, the integral may be found by completing the square:

$$\begin{aligned} \int_{-\infty}^{\infty} dt e^{-at^2 - bt} &= \int_{-\infty}^{\infty} dt \exp \left[-at^2 - bt - \frac{b^2}{4a} + \frac{b^2}{4a} \right], \\ &= \int_{-\infty}^{\infty} dt \exp \left[-a \left(t + \frac{b}{2a} \right)^2 + \frac{b^2}{4a} \right], \\ &= e^{b^2/4a} \int_{-\infty}^{\infty} du e^{-au^2}, \\ &= \sqrt{\frac{\pi}{a}} e^{b^2/4a}, \end{aligned} \quad (3.59)$$

where we have made the substitution $u \rightarrow t + b/2a$ in moving to the third line and used the well-known relation $\int_{-\infty}^{\infty} e^{-u^2} du = \sqrt{\pi}$ in moving to the last. Thus, (3.57) becomes

$$\widehat{\Psi}_\sigma(f) = \mathcal{F} \left\{ A e^{-2\pi i f_0 t} \exp \left[-\frac{t^2}{2\sigma^2} \right] \right\} = A \sqrt{2\pi} \sigma e^{-2\pi^2 \sigma^2 (f - f_0)^2}. \quad (3.60)$$

We're almost there! All we lack is the normalization A . We find that (3.54) provides

$$A^2 \int_{-\infty}^{\infty} e^{-\alpha^2 t^2 / \sigma^2} dt = A^2 \frac{\sqrt{\pi} \sigma}{\alpha} = 1 \quad \implies \quad A = \pi^{-1/4} \sqrt{\frac{\alpha}{\sigma}}, \quad (3.61)$$

and hence

$$\widehat{\Psi}_\sigma(f/\alpha) = \pi^{1/4} \sqrt{2\sigma\alpha} \exp \left[-2\pi^2 \sigma^2 (f/\alpha - f_0)^2 \right]. \quad (3.62)$$

Notice that this function is centered where $f/\alpha - f_0 = 0$ (*i.e.*, $f = \alpha f_0$), and so our scale relates directly to frequency! Combining the bulk of our results, the CWT for a Morlet wavelet is

$$\begin{aligned} \mathcal{W}\{x; \Psi_\sigma\}(\alpha, \tau) &= \frac{1}{\alpha} \mathcal{F}^{-1} \left\{ \hat{x}(f) \widehat{\Psi}_\sigma^*(f/\alpha) \right\}(\tau), \\ &= \pi^{1/4} \sqrt{\frac{2\sigma}{\alpha}} \mathcal{F}^{-1} \left\{ \hat{x}(f) \exp \left[-2\pi^2 \sigma^2 \left(\frac{f}{\alpha} - f_0 \right)^2 \right] \right\}(\tau). \end{aligned} \quad (3.63)$$

Consider now adjusting the time-like parameter σ . It is clear that in the limit $\sigma \ll 1$, the exponential is approximately unity, and $\mathcal{W}(\alpha, \tau) \sim x(\tau)$. Conversely, we see that when $\sigma \gg 1$, the exponential will tend

to zero everywhere $f \neq 0$, and $\mathcal{W}(\alpha, \tau) \sim \hat{x}(\alpha f)$. Thus, the CWT appears to be precisely what we desire: a smooth transition between the signal and its Fourier transform. There are, of course, some important and beautiful differences between the wavelet and Fourier approaches. For example, we may recall that the full-width half-maximum (FWHM) of a Gaussian function e^{-t^2/σ^2} is proportional to σ . A bit of rearrangement shows that the argument of the exponential in (3.63) may be written as $(-2\pi^2\sigma^2/\alpha^2)(f - \alpha f_0)^2$, which implies the effective Δf is proportional to α/σ ; thus the frequency resolution *decreases* with increasing scale α . Crucially, the opposite holds for Δt , which is proportional to σ/α . Compare this to the Fourier approach, which imposes a dedicated Δf and Δt based solely on our subinterval and chosen window!

Because this section is different from the rest of the technical report, we provide a self-contained code. First, we write a function to create and plot the CWT:

```
function CWT = cwt(x,dt,sigma)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Continuous wavelet transform
% x      -> time-series
% dt     -> sampling interval
% sigma  -> wavelet index (sigma>>1, CWT~fft, sigma<<1, CWT~x)

length_x = length(x); % Length of input vector
nyq = floor(length_x/2); % Index for Nyquist frequency

sample_vec = (0:length_x-1); % Time-series indices
freq_vec = (0:nyq-1)/(length_x*dt); % Build frequency space

f0 = 1/(length_x*dt); % Fundamental frequency

CWT = zeros(nyq); % Initialize

fft_x = fft(x); % Get FFT
fft_x = fft_x(1:nyq); % Keep only up to Nyquist

% Morlet wavelet in frequency space
Psi = @(a) (pi^0.25)*sqrt(2*sigma/a).*exp( -2*pi^2*sigma^2*( freq_vec/a - f0 ).^2);

for a=1:nyq
    CWT(a,:) = ifft(fft_x.*Psi(a));
end

% Plot it!
imagesc(sample_vec*dt,freq_vec,log10(abs(CWT)));
axis([0 (length_x-1)*dt 0 freq_vec(nyq)]);
title(['Scalogram (\sigma=' num2str(sigma) ')'],'fontsize',14,'fontweight','bold')
xlabel('Time (s)','fontsize',14,'fontweight','bold')
ylabel('Frequency (Hz)','fontsize',14,'fontweight','bold')
cbar = colorbar;
ylabel(cbar,'log_{10} |CWT|','fontsize',14,'fontweight','bold')
set(gca,'ydir','normal','fontsize',14);
```

To be completely accurate, we're plotting the logarithm of the *scalogram*,

$$\text{scalogram}(\alpha, \tau) = |\mathcal{W}\{x; \Psi_\sigma\}(\alpha, \tau)|^2. \quad (3.64)$$

In any case, we then analyze a quadratic chirp,

$$x(t) = \sin(2\pi 10t(1 + t^2/50)), \quad (3.65)$$

whose instantaneous frequency is given by

$$f_i(t) = \frac{1}{2\pi} \frac{d}{dt} [2\pi 10t(1 + t^2/50)] = 10 + \frac{3}{5}t^2 \text{ Hz}. \quad (3.66)$$

```

% Initialize
fS = 200;    % Sampling rate
dt = 1/fS;   % Sampling period
t = 0:dt:10; % Time vector
% Make signal
x = sin(2*pi*10*t.*(1+t.^2/50)); % Quadratic chirp
x = x + 0.25*(rand(1,length(t))-1); % Chirp + noise
% Analyze
W = cwt(x,dt,50);

```

The following figure presents the scalogram for six values of σ , and illustrates the variable nature of the CWT. For small σ (top left), the CWT is mostly a function of time, while it is mostly a function of frequency for large σ (bottom right).

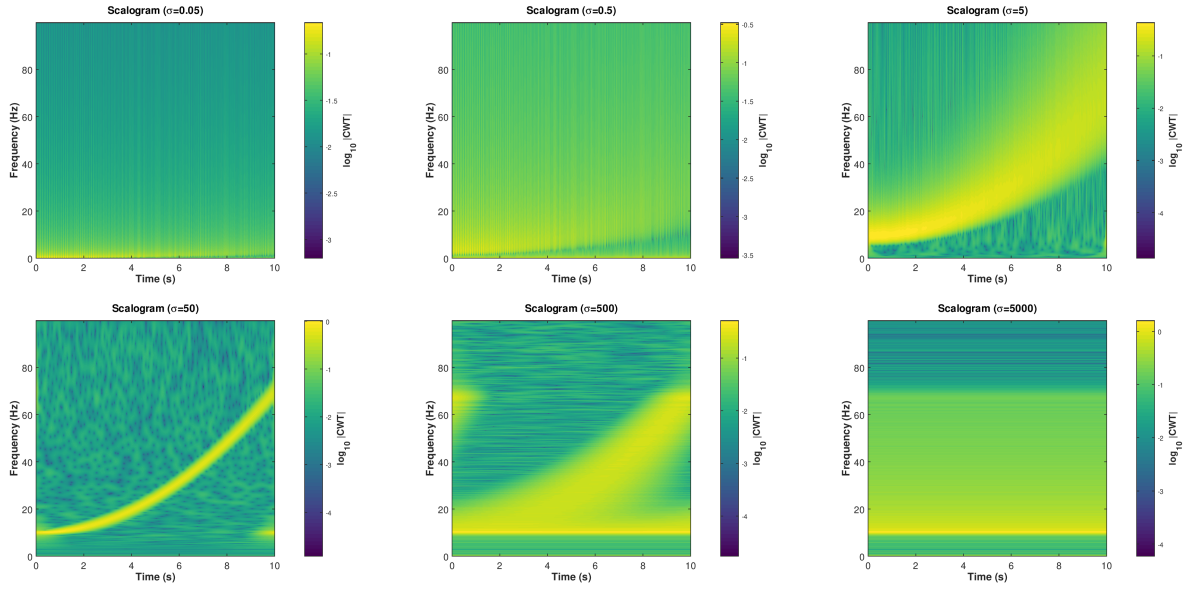


Figure 7: Series of scalograms for $\sigma = 5 \times 10^k$, where $k \in [-2, 3]$. Analyzed time-series is a quadratic chirp. Notice “sweet spot” for $\sigma = 50$.

To connect the CWT with a Fourier transform, we integrate it in time to yield

$$??? = \int_{-\infty}^{\infty} d\tau \mathcal{W}\{x; \Psi_{\sigma}\}(\alpha, \tau) \quad (3.67)$$

4 Practicum

4.1 pplk_bispec manual

Here we review the instructions and features of the MATLAB program given at the end of this document. The code is organized as follows (notice code sections are hyperlinked):

Main function

- [Parse arguments](#)
- [Check parameters](#)
- [Slice and repeat](#)

Subfunctions

- [Calculate bispectrum](#)
- [Apply LP filter](#)
- [Bispectrogram](#)
- [Plotting tool](#)
- [→Video](#)
- [→Figures](#)
- [“Utility” figure](#)
- [Label plots](#)
- [Choose current plot](#)
- [Scale frequencies](#)
- [Movie frame](#)
- [“Explore” feature](#)
- [Plot flag select](#)
- [Run demo](#)
- [Record audio](#)

```
W = pplk_bispec( samprate, res, subint, step, varargin )
```

Inputs

[Y] :: DATA TO BE ANALYZED. MAY BE:

- Time-series as a $2 \times \text{Nsig}$ (or $\text{Nsig} \times 2$) vector, where first row is time and second is **Nsig** data
- **Nseries** $\times$ **Nsig** (or **Nsig** $\times$ **Nseries**) vector, where **Nseries** is number of time-series
- Structure exported by **pplk_bispec**.

[samprate] :: SAMPLING RATE OF TIME-SERIES IN **hertz**

[res] :: DESIRED RESOLUTION OF FREQUENCY SPACE IN **hertz**

- Maximum resolution is **samprate/subint**.
- Automatic with **‘max’**.

[subint] :: SUBINTERVAL SIZE IN **samples**

- Must be in the interval **[1,Nseries]**.

[step] :: STEP SIZE IN **samples**

- Must be in the interval **[1,subint]**.

Options

[autoscale] :: CONTROLS AUTOSCALING IN PLOTTING TOOL (default = **false**)

- When **true**, colormaps will be scaled to data.

`[bispectro]` :: CALCULATE BISPECTROGRAM (default = `false`)

→ The form of (3.29) motivates the so-called *bispectrogram*.

$$\text{bispectrogram}(f) = \int_{-\infty}^{\infty} (\mathcal{B}_{x_1 x_2 x_3}(f', f) + \mathcal{B}_{x_1 x_2 x_3}(f, f') + \mathcal{B}_{x_1 x_2 x_3}(f', f - f')) df', \quad (4.1)$$

$$\text{bispectrogram}[k] = \sum_{j=0}^{N-1} (\mathcal{B}_{x_1 x_2 x_3}[j, k] + \mathcal{B}_{x_1 x_2 x_3}[k, j] + \mathcal{B}_{x_1 x_2 x_3}[j, k - j]). \quad (4.2)$$

→ Theoretically, should be spectrogram with uncoupled frequencies removed.

`[cmap]` :: CHOOSE COLORMAP (default = `'parula'`)

→ Adjusts default colormap in plotting tool.

`[dealias]` :: APPLY LOW-PASS FILTER (default = `false`)

→ Useful when high-frequency noise appears in signal.

→ Note: Once signal is discretized, true de-aliasing is not possible.

`[detrend]` :: REMOVE LINEAR TREND (default = `false`)

→ When `true`, each subinterval will be detrended according to

$$x_{\text{detrend}}[n] = x[n] - \frac{6n}{N(N^2 - 1)} \left(2 \sum_{m=0}^{N-1} mx[m] - (N + 1) \sum_{m=0}^{N-1} x[m] \right), \quad (4.3)$$

`[errlim]` :: ADJUST MEAN(DFT) TOLERANCE (default = `inf`)

→ For omitting dubious spectral content due to “bursty” behavior.

→ Sets DFT coefficients to zero in any subinterval where $\text{Mean}(\text{DFT}) > \text{errlim}$.

`[fscale]` :: SET FREQUENCY AXIS SCALE (default = `1`)

→ Feature of convenience for plotting.

→ For example, if `fscale = 1000`, then all labels will be in kHz.

`[justfft]` :: TURN OFF BISPECTRAL CALCULATION (default = `false`)

→ When `true`, will only output PSD and spectrogram — useful for preliminary analyses.

`[lilguy]` :: ADJUST COMPENSATION FOR NaN (default = `1e-6`)

→ Sets ε in (3.48).

`[plot]` :: PASS OUTPUT TO PLOTTING TOOL (default = `false`)

→ When `true`, plotting tool will launch automatically after calculation is complete.

`[plottype]` :: SELECT EMPHASIZED PLOTTABLE (default = `'bicoH'`)

→ Options are:

`'bicoH'`: Bicoherence spectrum

`'abs'`: Absolute value of bispectrum

`'real'`: Real part of bispectrum

`'imag'`: Imaginary part of bispectrum

`'angle'`: Biphase

`[sizewarn]` :: CONFIGURE WARNING FOR MATRIX SIZE (default = `true`)

→ If `true`, produces a warning dialog when frequency resolution exceeds $f_{\text{Nyq}}/1000$.

[smooth] :: SMOOTH DFT BY NUMBER OF SAMPLES (default = 1)

→ Values in DFT will be averaged; may be useful in noisy data.

[verbose] :: SUPPRESS STRUCTURE INFORMATION (default = false)

→ When true, output will only contain procedural information.

[window] :: CHOOSE WINDOW FUNCTION (default = 'hann')

→ Some/all of these may require Signal Processing Toolkit®:

'barthannwin', 'bartlett', 'blackman', 'blackmanharris', 'bohmanwin', 'chebwin', 'flattopwin', 'gausswin',
'hamming', 'hann', 'kaiser', 'nuttallwin', 'parzenwin', 'rectwin', 'taylorwin', 'triang', 'tukeywin'

→ Uses (3.20) with $\alpha = 2$ (the Hann window) if any error occurs.

Outputs

[W] :: Main output structure. Fields are:

[info] :: BASIC INFORMATION

→ Substructure containing the fields

'samprate': Sampling rate

'resolution': Frequency resolution

'subint': Subinterval size

'step': Step size

'window': Chosen window

'errlim': Mean(DFT) limit

'fscale': Axes scale

'lilguy': ε

'cmap': Selected colormap

'plotttype': Selected plottable

'autoscale': Scale to data range

'Nseries': Number of time-series

'Nsig': Number of data points

'Nint': Number of subintervals

'MatSize': Matrix size

[fv] :: FREQUENCY VECTOR

→ When Nseries = 1, is $1 \times \text{MatSize}$ vector given by $\text{fv}[k] = (k - 1)(\text{resolution}/\text{fscale})$.

→ When Nseries > 1, is $1 \times (\text{MatSize} + 1)/2$ vector given by $\text{fv}[k] = (k - 1)(\text{resolution}/\text{fscale})$.

[ff][†] :: FULL FREQUENCY VECTOR

→ $1 \times \text{MatSize}$ vector which mirrors fv across $k = 0$.

[ft] :: (SQUARE ROOT OF) POWER SPECTRAL DENSITY

→ When Nseries = 1, is $1 \times \text{MatSize}$ vector given by $\text{ft}[k] = \sqrt{\text{PSD}[k - 1]}$. [See (3.47)].

→ When Nseries > 1, is $\text{Nseries} \times (\text{MatSize} + 1)/2$ vector given by $\text{ft}[n, k] = \sqrt{\text{PSD}\{x_n\}[k - 1]}$.

[bs][‡] :: BISPECTRUM

→ $\text{MatSize} \times \text{MatSize}$ matrix given by $\text{bs}[j, k] = \mathcal{B}_{x_1 x_2 x_3}[j - 1, k - 1]$, as defined in (3.48).

[bc][‡] :: BICOHERENCE SPECTRUM

→ $\text{MatSize} \times \text{MatSize}$ matrix given by (3.48).

[bp][†] :: BIPHASE PROXY
→ $1 \times \text{Nint}$ vector given by $\text{bp}[m] = \sum_{j,k} \hat{x}_1[j]_m \hat{x}_2[k]_m \overline{\hat{x}_3[j+k]_m}$.
→ Real part of bispectrum is related to biphase modulation.

[bg]^{†*} :: BISPECTROGRAM
→ $\text{MatSize} \times \text{Nint}$ matrix given by $\text{bg}[m, k] = \text{bispectrogram}[m-1, k-1]$ [see (4.2)].

[sg] :: SPECTROGRAM
→ When **Nseries** = 1, is $\text{MatSize} \times \text{Nint}$ matrix given by $\text{sg}[m, k] = \hat{x}[k-1]_{(m-1)}$ [see (3.48)].
→ When **Nseries** > 1, is $(\text{MatSize} + 1)/2 \times \text{Nint} \times \text{Nseries}$; $\text{sg}[m, k, n] = \hat{x}_n[k-1]_{(m-1)}$.

[tv] :: TIME VECTOR
→ $1 \times \text{Nint}$ vector given by $\text{tv}[m] = (m-1)(\text{step}/\text{samprate})$.
→ For convenience in plotting spectrogram.

[er] :: ERROR VECTOR
→ $\text{Nseries} \times \text{Nint}$ vector given by $\text{er}[n, m] = \text{mean}(\hat{x}_n[k]_m)$.

[†] Appears when **Nseries** > 1.

[‡] Available only if **justfft** = **false**.

* Available only if **bispectro** = **true**.

Usage

W = **pplk_bispec**

- When no inputs are supplied, the user can choose between a ‘demo’ and ‘record’ mode.
- Record mode allows user to record in 10 seconds of audio @ 8192 Hz., can output directly to array and/or WAV file.

pplk_bispec(**W**)

- If input is detected to be a structure produced by **pplk_bispec**, user is given option to plot data or export video (AVI). Please study Figure 8 to better understand the plotting tool’s workflow.

W = **pplk_bispec**(**Y**)

- Assumes **Y** is a $2 \times \text{Nsig}$ [$\text{Nsig} \times 2$] array w/ **Y**(1,:) [**Y**(:,1)] as time values and **Y**(2,:) [**Y**(:,2)] as data.
- Automatically detects sampling rate, and chooses subinterval as min(next smallest power of 2, 512).

W = **pplk_bispec**(**Y**, **samprate**, **res**)

- Assumes **Y** is a $\text{Nseries} \times \text{Nsig}$ [$\text{Nsig} \times \text{Nseries}$] array w/ **Y**(i,:) [**Y**(:,i)] as data ($1 \leq \text{Nseries} \leq 3$)
- Automatically chooses subinterval and step size.

W = **pplk_bispec**(**Y**, **samprate**, **res**, **subint**, **step**)

- Assumes **Y** is a $\text{Nseries} \times \text{Nsig}$ [$\text{Nsig} \times \text{Nseries}$] array w/ **Y**(i,:) [**Y**(:,i)] as data ($1 \leq \text{Nseries} \leq 3$)
- Uses default values for additional options (window, colormap, etc.).

W = **pplk_bispec**(**Y**, **samprate**, **res**, **subint**, **step**, ‘P1’, P1Val, ..., ‘Pj’, PjVal)

- Assumes **Y** is a $\text{Nseries} \times \text{Nsig}$ [$\text{Nsig} \times \text{Nseries}$] array w/ **Y**(i,:) [**Y**(:,i)] as data ($1 \leq \text{Nseries} \leq 3$)
- Sets parameters P1, ..., Pj to values P1Val, ..., PjVal, respectively.
- See above (§4.1) for further explanation of params/values.
- Use of unknown options will trigger warning and reversion to default.

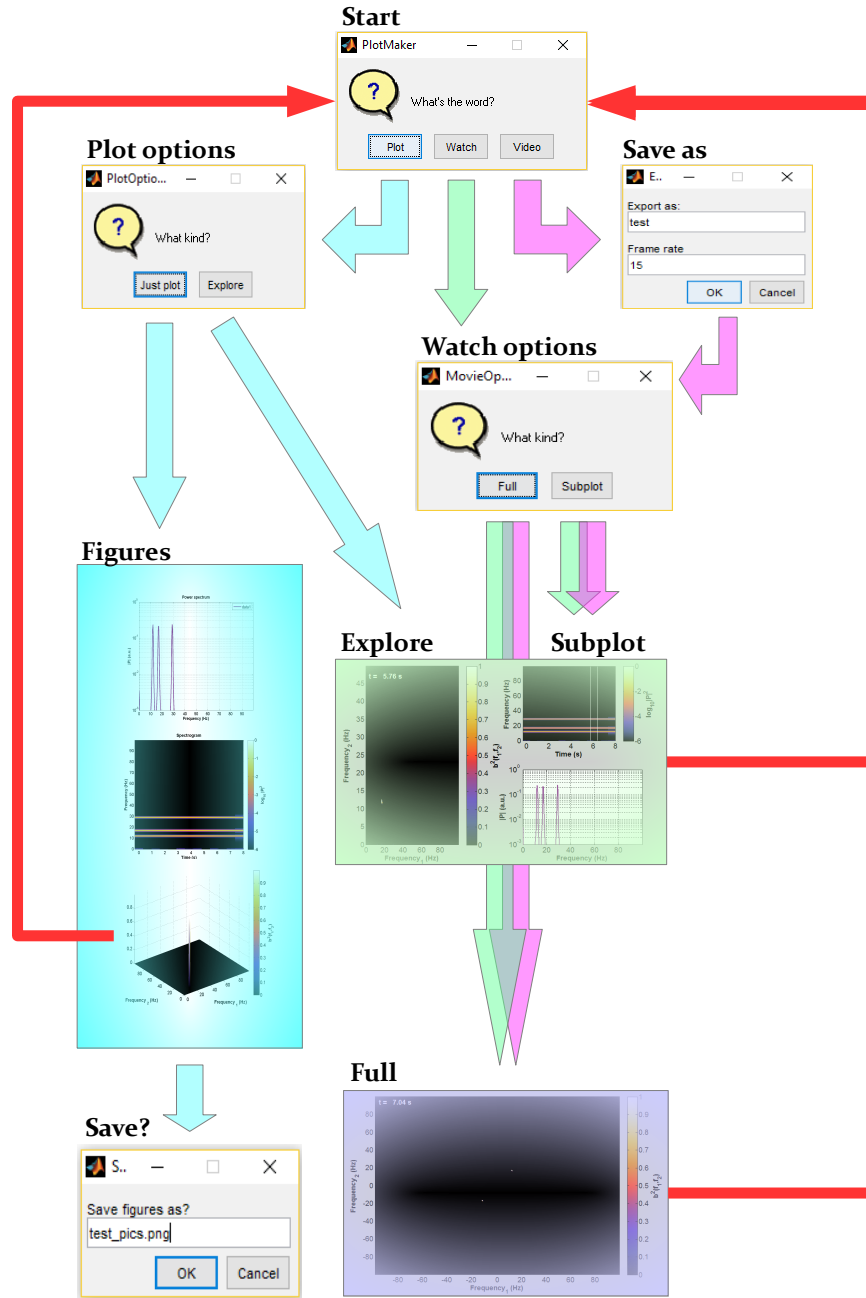


Figure 8: Block diagram of workflow of plotting tool. Bold red lines represent figures whose `CloseRequestFcn` is a callback to the PLOT, WATCH, VIDEO prompt. Closing a window which is not tied to the root prompt, or the PLOT, WATCH, VIDEO prompt itself exits the plotting tool.

4.2 Example MATLAB analysis

Let's sketch out a simple use of `pplk_bispec.m`. First, we create a 10 second time vector, in steps of 50 milliseconds (*i.e.*, sampled at 200 Hz):

```
fs = 200; % Sampling rate
dt = 1/fs; % Sampling period
t = 0:dt:10; % Time vector
```

Now, we create three sine waves at 12, 17, and 29 Hz:

```
x1 = sin(2*pi*12*t); % 12 Hz
x2 = sin(2*pi*17*t); % 17 Hz
x3 = sin(2*pi*29*t); % 29 Hz
```

To assure ourselves we have reasonable data, let's make a pretty plot of it:

```
plot(t,x1,t,x2,t,x3,'marker','o','linewidth',2); % Plot
xlim([0 0.1]); % Limit window
xlabel('Time (s)','fontsize',14,'fontweight','b'); % X label
ylabel('Amplitude (a.u.)','fontsize',14,'fontweight','b'); % Y label
legend('12 Hz','17 Hz','29 Hz'); % Legend
set(gca,'fontsize',14,'fontweight','b'); % Change unaffected fonts
grid on
```

As a first analysis, load the time-vector and the 29 Hz sine wave as a single 2×2001 vector:

```
Y = [t' x3']; % Concatenate to create new vector with 2 rows
q = pplk_bispec(Y); % Run program
```

Assuming the routine returns no errors, we should be greeted with a survey of various parameters. For instance, the program should report `samprate`: 200, which was deduced from the time-vector. To plot the Fourier components vs. frequency, we can use

```
semilogy(q.fv,q.ft,'linewidth',2); % Log plot
xlabel('Frequency (Hz)','fontsize',14,'fontweight','b'); % X label
ylabel('P^{1/2}','fontsize',14,'fontweight','b'); % Y label
set(gca,'fontsize',14,'fontweight','b'); % Change unaffected fonts
grid on
```

An easier way to visualize our data is with the built-in plotting tool. To do so, use the outputted structure as an input to bring up the PLOT, WATCH, VIDEO prompt (select PLOT → JUST PLOT),

```
q = pplk_bispec(q);
```

Three new figures should have appeared, along with a prompt to save said figures. After either saving or cancelling, the most recent figure should be the bicoherence spectrum. Minimize this for the moment. The present figure should then be a spectrogram, reporting a single frequency (29 Hz) over a 10 second duration. This is a good sign. Close this figure to find a semilog-scale plot of the (square root of the) power spectrum. If there is a single peak around 29 Hz, all is right in the world. Close this, and restore the minimized figure of the bicoherence spectrum. Now, theory tells us that there should be no bicoherent features in this system, as only a single oscillation exists in the time-series. Nevertheless, we should be observing what looks to be a strong peak near $(f_1, f_2) = (29, 0)$ Hz. Notice that the limit of the z -axis is very small — we have simply scaled b^2 such that even tiny features appear to be important. In any case, use the following keyboard commands to switch between different plots:

Command	Plotted quantity	Name
SHIFT + B	b^2	Bicoherence spectrum
SHIFT + A	$ \mathcal{B} $	Absolute value of bispectrum
SHIFT + R	$\text{Re}(\mathcal{B})$	Real part of bispectrum
SHIFT + I	$\text{Im}(\mathcal{B})$	Imaginary part of bispectrum
SHIFT + P	$\tan^{-1}(\text{Im}\mathcal{B}/\text{Re}\mathcal{B})$	Biphase

After exploring the numerical artifacts posing as bispectral features (take note of the z -axis scale!), close the figure, which will return us to the PLOT, WATCH, VIDEO prompt. Close this to return to the workspace. Now, make a new time-series, and load it into the routine with explicit parameters (select PLOT → EXPLORE when prompted):

```
Y = x1 + x2 + x3; % Satisfies f1+f2=f3
q = pplk.bispec(Y,200,'max',512,128,'plot',true);
```

A new figure should appear displaying a bicoherence spectrum, a spectrogram, and a power spectrum. By clicking on some point within the spectrogram, we will:

- **Select a slice of the spectrogram, denoted with white lines,**
- **View the power spectrum of that slice,** and
- **View the bicoherence spectrum calculated at that slice.**

This behavior may be repeated *ad nauseum*, and the left-hand plot may be changed with the SHIFT + modifiers as in the “utility plot” (a slice of spectrogram must be selected to refresh). Be sure to observe the single (now strong) peak in the bicoherence spectrum at $(f_1, f_2) = (12, 17)$ Hz. Close this figure and exit the PLOT, WATCH, VIDEO prompt to return to the workspace. For comparison, let’s add a phase modulation to our 12 Hz oscillation and repeat this analysis

```
y1 = sin(2*pi*12*t + 2*t); % Linear phase modulation
Y = y1 + x2 + x3;
q = pplk.bispec(Y,200,'max',512,128,'plot',true);
```

Again using the EXPLORE tool, change to the imaginary part of the bispectrum with SHIFT + I, and move through the spectrogram. You might notice the color scale is oscillating. To be sure, exit the EXPLORE window, returning to the PLOT, WATCH, VIDEO prompt. Now, select WATCH → SUBPLOT to animate the movement through the spectrogram, and watch the scale oscillate as time progresses. Additionally, it may be useful to look at the bicoherence spectrum (exit, PLOT → JUST PLOT) to notice the single peak has been significantly diminished — well in line with theory. Now, consider a quadratic nonlinearity:

```
Y = x1 + x2 + x1.*x2; % Notice nonlinear term
q = pplk.bispec(Y,200,'max',512,128,'plot',true);
```

Notice the two peaks at the sum and difference frequencies $(f_1, f_2) = (12, 17)$ Hz and $(f_1, f_2) = (12, 5)$ Hz. After exploring these figures, try a nonlinear term where one factor has a linear phase:

```
Y = y1 + x2 + y1.*x2; % Nonlinear with PM
q = pplk.bispec(Y,200,'max',512,128,'plot',true);
```

Unlike our first experiment with phase modulation, this analysis yields two strong peaks, again confirming our theoretical results. Let us now address noise, and input

```
y1 = x1 + 0.25*rand(1,length(t)); % Add noise
Y = y1 + x2 + y1.*x2; % Same nonlinear term
q = pplk.bispec(Y,200,'max',512,128,'plot',true,'justfft',true);
```

This should have completed much faster than the other inputs. Additionally, the use of ‘justfft’ suppresses the utility plot, so it is easier to get back to the workspace. In any case, the white noise is clearly visible in the power spectrum and spectrogram. Adjust the time resolution and apply a low-pass filter with

```
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'dealias',true,'justfft',true);
```

Suppress output, and set the frequency axes to kHz using

```
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'justfft',true,'verbose',false,'fscale',1000);
```

Let’s explore the bispectrogram. Create a fourth oscillation which does not satisfy a sum-frequency condition, and rerun the program:

```
x4 = sin(2*pi*20*t) % New frequency
Y = y1 + x2 + x3 + x4; % Add new and keep noisy term
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'bispectro',true);
```

Using the PLOT → JUST PLOT option, compare the spectrogram with the bispectrogram. There should be a “missing” frequency component (at 20 Hz) in the latter, as this frequency is not participating in the three-wave interaction. Now, add a linear component to your data with

```
y1 = x1 + 10*t; % Add linear trend
Y = y1 + x2 + x3;
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'justfft',true);
```

Use the EXPLORE option to look through the spectrogram, noticing the peak in the 0 Hz (DC) bin. Reduce it with

```
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'justfft',true,'detrend',true);
```

Again, use the EXPLORE tool to look through the spectrogram, finding the DC bin relatively empty. If you also observe the PSD with PLOT → JUST PLOT, you will notice that there remains a significant DC component, due to the effect of zero-padding on the last many subintervals. Now, consider a linear component to the instantaneous frequency of x1, by using

```
y1 = sin(2*pi*12*t.*(1+t/25)); % Linear frequency chirp
Y = y1 + x2 + x3;
q = pplk.bispec(Y,200,'max',512,256,'plot',true,'justfft',true);
```

Use the WATCH → SUBPLOT tool to watch the frequency change over time, noticing the low time-resolution. Increase this resolution with

```
q = pplk.bispec(Y,200,'max',512,32,'plot',true,'justfft',true);
```

Again using WATCH → SUBPLOT, watch the single bicoherent feature slowly disappear as the frequency chirp removes the sum-frequency condition. Instead, compare with

```
y2 = sin(2*pi*17*t.*(1+t/25)); % 17 Hz
y3 = sin(2*pi*29*t.*(1+t/25)); % 29 Hz
Y = y1 + y2 + y3;
q = pplk.bispec(Y,200,'max',512,32,'plot',true);
```

We did not use the ‘**justfft**’ option because we want to see the final bicoherence spectrum (use PLOT → JUST PLOT to get to the utility figure). Notice that instead of a single feature, the bicoherent peak has been “smeared out” linearly. Return to the PLOT, WATCH, VIDEO prompt and use WATCH → SUBPLOT or WATCH → FULL to observe the bispectral peak move through bi-frequency space. Notice it remains strong (~ 1) because the three frequencies chirp together, maintaining the sum-frequency condition throughout. For completeness, repeat the analysis of a quadratic nonlinearity, this time retaining the chirps:

```
Y = x1 + y2 + x1.*y2; % Nonlinear w/ chirp
q = pplk.bispec(Y,200,'max',512,32,'plot',true);
```

As we might expect, the results are the same — nonlinearly coupled oscillations preserve the sum-frequency condition even when phase or frequency modulated. The PLOT → JUST PLOT option is particularly interesting, as it shows the linear smearing of the sum and difference frequencies in the bicoherence spectrum. At any rate, return to the workspace and perform a cross-bicoherence analysis with

```
Y = [x1' x2' x3']'; % 3x2001 matrix
q = pplk.bispec(Y,200,'max',512,32,'plot',true);
```

This should have taken a bit longer to calculate than the auto-bicoherence spectra. [If the render time felt “too long”, reduce **step** to 64, 128, etc.] Use PLOT (noting the absence of the JUST PLOT option) to view our results. When reviewing the plots, take note of (1) the full bi-frequency space and (2) the “conjugate” point at $(f_1, f_2) = (-12, -17)$ Hz. For clarity, use SHIFT + I to look at the imaginary part of the bispectrum and confirm the two peaks do in fact satisfy $\mathcal{B}_{x_1 x_2 x_3}(f_1, f_2) = \mathcal{B}_{x_1 x_2 x_3}(-f_1, -f_2)$. Additionally, it may be fun to permute the time-series ($Y = [x2' x1' x3']'$; , say). Something more interesting is:

```
Y = x1 + x2 + x3;
Z = [Y' Y' Y']';
q = pplk.bispec(Z,200,'max',512,32,'plot',true);
```

This is the full auto-bispectrum, and should display 12 coherent points in the full bi-frequency space. If using the utility plot (or any other interactive figure), notice that SHIFT + I clearly shows the conjugacy as described in Figure 5. What if we used our frequency-chirped time-series? Find out with

```
Y = y1 + y2 + y3;
Z = [Y' Y' Y']';
q = pplk.bispec(Z,200,'max',512,32,'plot',true);
```

Use the WATCH tool (notice there is no FULL, SUBPLOT prompt) to observe the 12 points move away from the origin as time progresses. Conveniently, the SHIFT + modifiers can be used while watching. Also,

```
Y = y1 + y2 + y1.*y2;
Z = [Y' Y' Y']';
q = pplk.bispec(Z,200,'max',512,32,'plot',true);
```

This has the same behavior of the last plot — points moving “away from the origin” — but there are now 24 points representing the sum and difference frequencies. Finally, we will look at the full auto-bicoherence spectrum of a nonlinearly coupling between a static oscillation and a frequency chirped one:

```
Y = x1 + y2 + x1.*y2;
Z = [Y' Y' Y']';
q = pplk.bispec(Z,200,'max',512,32,'plot',true);
```

If this is interesting, use the VIDEO tool to export an AVI! [May require some code un-commenting to work in newer versions of MATLAB.] This concludes the pplk_bispec tutorial.

4.3 Source code

```
function W = plk.bispec(Y,samprate,res,subint,step,varargin)
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Bicoherence analysis via the short-time Fourier transform (STFT)
%      o\ /\ /\ /\ /\ /\ /\ .... \ /\ /\ /\ /\ /\ /\ /\ o
%    by G. Riggs for Plasma Physics Laboratory - Koepke
%
%      -----
%            (c) 2021 West Virginia University
%      -----
%          contact me :: gariggs@mix.wvu.edu
%      o\ /\ /\ /\ /\ /\ /\ /\ ^--^ \ /\ /\ /\ /\ /\ /\ /\ o
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% The Bispectrum
% B_xyz(f1,f2) = < X(f1)Y(f2)Z(f1+f2)* >, where x,y,z are time series with
% corresponding Fourier transforms X,Y,Z, and <...> denotes averaging.
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% The (squared) Bicoherence spectrum
% b^2_xyz(f1,f2) = |B_xyz(f1,f2)|^2
% -----
%      ( <|X(f1)Y(f2)|^2> <|Z(f1+f2)|^2> + eps ),
% where eps is a small number meant to prevent 0/0 = NaN catastrophe
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Inputs
% -----
% Y           -> time-series {or structure}
% samprate    -> sampling rate [Hz]
% res         -> desired frequency resolution* [Hz]
% subint      -> subinterval size** [samples]
% step        -> step size for Welch method [samples]
% -----
% additional options... (see below for instructions)
% -----
% window      -> select window function [default :: "hann" (see @window)]
% justfft     -> true for just spectrogram [default :: false]
% errlim      -> mean(fft) condition [default :: inf]
% fscale      -> scale for plotting frequencies [default :: 1]
% dealias     -> applies antialiasing (LP) filter [default :: false]
% bispectro   -> computes bispectrogram [default :: false]
% smooth      -> smooths FFT by n samples [default :: 1]
% plot        -> start plotting tool when done [default :: false]
% lilguy      -> set epsilon [default :: 1e-6]
% sizewarn    -> warning for matrix size [default :: true]
% cmap        -> adjust colormap [default :: parula]
% plottype    -> set desired plottable [default :: 'bicoh']
% autoscale   -> autoscaling in figures [default :: false]
% verbose     -> allow printing of info structure [default :: true]
% detrend     -> remove linear trend from data [default :: false]
% * "max" is available for convenience!
% ** For efficiency use powers of 2 and max resolution!
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Output
% -----
% Structure with fields:
% info -> information from inputs/run params
% fv   -> frequency vector [Hz]
% ff*  -> "full" frequency vector (-Nyq , Nyq)
% ft   -> FFT magnitudes
% bs** -> bispectrum
% bc** -> bicoherence spectrum
```

```

% bp** -> biphas proxy (sum of bispectrum)
% bg*** -> bispectrogram
% sg -> spectrogram
% tv -> time vector
% er -> evolution of mean(FFT)
% * Appears if number of time-series (Nseries) exceeds 1
% ** Assuming JustFFT == false
% *** Assuming Bispectro == true
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Usage
% -----
% "W = pplk.bispec"
% When no inputs are supplied, the user can choose between a 'demo' and
% 'record' mode. Demo mode presents the basic features; record mode allows
% user to record in 10 seconds of audio @ 8192 Hz, then analyze!
% If recording, can output directly to 2xn array and/or WAV file.
% -----
% "pplk.bispec(W)"
% If input is detected to be a structure produced by @pplk.bispec, user
% is given option to plot data or export video (AVI). See below for info.
% -----
% "W = pplk.bispec(Y)"
% Assumes Y is a 2xn [nx2] array w/ Y(1,:) [Y(:,1)] as time values
% and Y(2,:) [Y(:,2)] as data. Automatically detects sampling rate,
% and chooses subinterval as min(next smallest power of 2 , 512).
% -----
% "W = pplk.bispec(Y,samprate,res)"
% Assumes Y is a mxn [nxm] array w/ Y(i,:) [Y(:,i)] as data (1 <= m <= 3).
% Automatically chooses subinterval and step size.
% -----
% "W = pplk.bispec(Y,samprate,res,subint,step)"
% Assumes Y is a mxn [nxm] array w/ Y(i,:) [Y(:,i)] as data (1 <= m <= 3).
% Uses default values for additional options (window, colormap, etc.).
% -----
% "W = pplk.bispec(Y,samprate,res,subint,step,P1,P1Val,...,Pj,PjVal)"
% Assumes Y is a mxn [nxm] array w/ Y(i,:) [Y(:,i)] as data (1 <= m <= 3).
% Sets parameters P1,...,Pj to values P1Val,...,PjVal, respectively.
% See above ("Inputs") for further explanation of params/values.
% Use of unknown options will trigger warning and reversion to default.
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Plotting
% -----
% After the FFT coefficients have been calculated, the output structure can
% be loaded into the program for analysis. 3 options are given:
% -----
% 1) Plot
% If Nseries == 1, further options are given:
% -> Just plot :: Go to plots
% -> Explore :: Explore spectrogram and corresponding line-outs,
% and bicoherence spectrum
% Else, power spectrum, spectrogram, [bicoherence spectrum, and
% bispectrogram are produced]*. Option to save as PNG is given.
% * These appear if JustFFT == false and Bispectro == true.
% -----
% 2) Watch
% For previewing the animation by the 'Video' button.
% If Nseries == 1, further options are given:
% -> Full :: Only 1st quadrant bicoherence spectrum
% -> Subplot :: Same figure as 'Explore' option
% Else, full bi-frequency domain (-Nyq < f < Nyq) is shown for
% cross-bicoherence spectrum.
% -----
% 3) Video

```

```

% Same as 'Watch' with addition of prompt for naming AVI file.
% - - - - -
% With the exception of 'Just plot' (or 'Plot' when Nseries > 1),
% closing the figure window will return the user to [Plot,Watch,Video]
% prompt. To exit plot tool, user must close this prompt w/ no selection.
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Tips & Tricks
% - - - - -
% While the auto-bispectral (Nseries ==1) calculation is well-optimized,
% a general cross-bispectral calculation is absolutely not. Please be
% patient when Nseries == 2 or 3, hopefully future versions will be faster!
% - - - - -
% Sometimes it's difficult to nail down the best resolution/subinterval/
% step size to maximize some feature of your plot. Don't be afraid to
% finagle! Often it will require two different renders for perfection.
% Using 'max' resolution, while convenient, can be overkill for b^2.
% - - - - -
% It is *much* faster to calculate a spectrogram than to produce the
% bispectrum/bicoherence spectrum of data. If processing time is brutal,
% try "pplk.bispec(...,'justfft',true)". Then, bicoherence spectra are
% available through the 'Explore' option in the plotting tool. In fact,
% this is an intended feature, as keeping each frame is taxing on memory.
% A video is generally less unwieldy than 1000 750x750 matrices, say.
% - - - - -
% A possible benefit of and potential curse to the plotting tool is that
% it stores the data structure in its 'UserData' property. Technically,
% one might immediately save the outputted structure to a MAT file after
% the plotting tool is invoked, freeing up workspace memory. Additionally,
% this means that multiple instances of 'Explore', 'Watch', or 'Video'
% figures are self-contained applets (though using multiple 'Watch' or
% 'Video' windows is *not* recommended). Unfortunately, this is a clear
% waste of RAM, as the data structure appears in both the figure and
% workspace memory. Let me know if you encounter any memory leaks!
% - - - - -
% Finally, much care has gone to the removal of bugs with try/catch
% blocks. There are few things worse than encountering a persistent glitch
% in homebrew software, only to find that it was caused by some tiny/huge
% oversight on the part of the programmer. That being said, there is
% surely a menagerie of malevolent instructions buried in this program.
% My hope is that they reside in the "bells & whistles" (i.e., the
% plotting and video exporting functions), but I cannot be more than 99.9%
% certain the engine of this program has been absolved from errancy. Please
% inform me if you find one, whether it is glaring or glancing. Three
% cheers for the unending, collaborative monster we call "science"!
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Version History
% 7/26/2021 -> v1.3 => Changed default colormap to 'parula', cleaned up
% expression for detrend, and fixed subinterval warning ['%s'-> '%d']
% 7/8/2021 -> v1.2 => Removed negative sampling frequency issue for
% auto-detected time-series, added detrending option, fixed check for
% properly form
% 7/5/2021 -> v1.1 => Bug fixes (window toolbox error), reduction in
% movie.slice(...) inputs, window (lines) on spectrogram now correct,
% validity checks for both subint and step.
% 7/4/2021 -> v1.0 => "Release" %^o/-<
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
fprintf(' Checking inputs...');
%% Parse inputs
if nargin==0 % Offer demonstration/record
    qwer = questdlg('Hi there! What's s up?', 'StartUp', 'Demo', 'Record', 'Demo');
    pause(eps)
    switch qwer
        case 'Demo'

```

```

        W = run_demo;           % Go to demo
    case 'Record'
        dats = record_demo;    % Get (and possibly save) WAV file
        qwer = questdlg('What about...', '', 'Export', 'Analyze', 'Export');
        pause(eps)             % This pause helps prevent odd behavior
        if isequal(qwer, 'Analyze')
            W = pplk_bispec(10*dats(2,:), 44100/4, 'max', 2048, 128, ...
                'window', 'flattopwin', ...
                'justfft', true, ...
                'plot', true, ...
                'fscale', 1e3);
        else
            % User has bailed on analysis
            W = dats;
        end
    otherwise
        % Send empty matrix and leave!
        fprintf('done.\n'); W = [];
        return
    end
    return
end
try
    chk = size(Y); % Get data dimensions
    if chk(1)>chk(2) % Check if column vector
        Y = Y'; % As data is real-valued, adjoint is transpose
        chk = fliplr(chk); % All better!
    end
    Nseries = chk(1);
    if Nseries>3
        error('Bispec:Input', 'Invalid input!\nMaximum time-series is 3...\nSee "help ...
            pplk_bispec"');
    end
    Nsig = chk(2);

    if nargin==1 % Full automatic
        if isstruct(Y) % Check if structure
            W = Y; % Yes? Just plot stuff then!
            try
                plot_results(Y);
            catch plotbeef
                switch plotbeef.identifier
                    case 'MATLAB:capturescreen:FigureDestroyedDuringGetFrame'
                        warning('Bispec:killScreen', 'Try not to do that!')
                    otherwise
                        error('Bispec:Input', 'Invalid input!\nStructure must be output by ...
                            @pplk_bispec...')
                end
            end
            fprintf('done\n');
            return
        else
            tchk = diff(Y(1,:));
            if Nseries==2 % If not, see if properly formatted
                error('Bispec:Input', 'Invalid input!\nMust have time-vector...\nSee "help ...
                    pplk_bispec"');
            elseif var(tchk/tchk(1))>1
                varchk = diff(Y(2,:));
                if var(varchk/varchk(1))<1
                    Y = flipud(Y);
                    warning('Bispec:Input', 'Please use time-vector as first row or column...');
                else
                    % This is not the best way...
                    error('Bispec:Input', 'Invalid input!\nTime-vector but be evenly ...
                        sampled...\nSee "help pplk_bispec"');
                end
            end
            end
            samprate = abs(1/(Y(1,2) - Y(1,1)));

```

```

        Nseries = 1; % Readjust params to work properly
        Y = Y(2,:);
        res = 'max'; % Assumes maximum resolution
    end
end
if nargin<=3 % Auto subinterval
    npow = log2(Nsig);
    if mod(npow,1)==0 % Check exact power of 2
        ex = npow-1;
    else
        ex = floor(npow);
    end
    subint = min(512,2^ex); % Try to keep it small, I guess?
    step = subint/4; % Seems reasonable
end
subint = floor(abs(subint));
if subint==0 || subint>Nsig % Check subinterval length
    subint = min(512,Nsig);
    warning('Bispec:subintWarn','Subinterval too large for time-series... Using %s.',subint)
end
if isequal(res,'max') % Check max res option
    res = samprate/subint; % Maximum resolution
end
nf = floor(samprate/res); % Subinterval required for resolution
if nf>subint
    warning('Bispec:resWarn','Subinterval too small for resolution... Using maximum.')
    res = samprate/subint; % Really hate repeating code, but...
end
step = floor(abs(step));
if step==0 || step>subint % Nobody likes infinite loops... loops...
    step = floor(subint/4);
    warning('Bispec:Input','Step must be nonzero and less than subint... Using %d.',step)
end
% - - - - -
%% Defaults & varargin stuff
windoe = 'hann';
JustFFT = false;
errlim = inf;
fscale = 1;
AntiAlias = false;
Bispectro = false;
SmoothFFT = 1;
PlotIt = false;
lilguy = 1e-6;
SizeWarn = true;
CMap = 'parula';
PlotType = 'bicoH';
AutoScale = false;
Verbose = true;
Detrend = false;
% There's definitely a smarter way to do this, but I kind of dig it!
% Six nested loops/conditionals is a good time ;^)
% I know that lower(options) *should* = inputs, but eh
options = {'windoe','JustFFT','errlim','fscale','AntiAlias','Bispectro',...
    'SmoothFFT','PlotIt','lilguy','SizeLim','CMap','PlotType',...
    'AutoScale','Verbose','Detrend'};
inputs = {'window','justfft','errlim','fscale','dealias','bispectro',...
    'smooth','plot','lilguy','sizewarn','cmap','plottype',...
    'autoscale','verbose','detrend'};
if nargin>5
    for i=1:2:(nargin-5)
        switch lower(varargin{i})
            case inputs
                for j=1:length(options)
                    if isequal(inputs{j},lower(varargin{i}))

```



```

        cl = eval(sprintf('class(%s)',options{j}));
        if isequal(cl,class(varargin{i+1}))
            eval(sprintf('%s = varargin{i+1};',options{j}));
        else
            warning('Bispec:errOption','\n%s" must be a %s... Using ...
                    default value.',...
                    varargin{i},cl)
        end
    end
    end
    end
    otherwise
        warning('Bispec:unknownOption','\n%s" is not a known option...',varargin{i})
    end
end
end
catch beef % Assess errors and proceed
    switch beef.identifier
        case 'MATLAB:badsubscript'
            warning('Bispec:missingOption','\n%s" value missing... Using default.',varargin{i})
        case 'MATLAB:dimagree'
            error('Bispec:Input','\nInvalid input!\nSee help pplk.bispec...')
        otherwise
            error(beef.message);
    end
end
fprintf('done.\n');
% Whew! Finally getting to the...
% -----
%% Preliminaries
if nf>1000 && ~SizeWarn;           % Resolution exceeds Nyq/1000
    str = sprintf('nf = %d',nf);
    qwer = questdlg({'FFT elements exceed 1000...';str;'Continue?'},...
        'FFTWarning','Absolutely!','No way...','No way...');
    pause(eps)
    if isequal(qwer,'No way...')
        fprintf('\nOperation terminated by user.\n')
        W = []; return           % Bail if that seems scary!
    end
end
lim = floor(nf/2);                % lim = (Nyquist/res) is matrix size

fprintf(' Checking length...');
tailerror = mod(Nsig,subint);      % Assess tail errors with modulo
Nextra = 0;
if tailerror~=0
    Y = [Y zeros(Nseries,subint-tailerror)];
    Nextra = subint-tailerror;      % Problem? Tack on some zeros...
    Nsig = Nsig + Nextra;          %...now it's this long
end
fprintf('done.\n\tNsig = %d + %d\n',Nsig-Nextra,Nextra);

try
    win = window(windoe,nf);        % Use MATLAB's windowing function
catch winbeef
    warning('Bispec:wrongWindow','\n%s" window unknown... Using Hanning.',windoe)
    win = sin(pi*(0:nf-1)/(nf-1));  % Hann is just half-cycle sine^2
end

f = (0:nf-1)*samprate/nf;          % Creates frequency domain
M = 1 + (Nsig - subint)/step;      % Number of stepped subintervals

if AntiAlias
    fprintf(' Removing alias...');  %#ok<*UNRCH>
    for k=1:Nseries
        Y(k,:) = dealias(Y(k,:));  % Apply to each time-series
    end
end

```

```

    end
    fprintf('done.\n')
end

% -----
%% "Preallocating for speed" ;^)
MatSize = lim;
if Nseries>1
    MatSize = 2*lim-1; % Must account for full domain
    f = [-fliplr(f(1:lim)) f(2:lim)];
end
if ~JustFFT
    e = zeros(MatSize); e12 = e; e3 = e; % Bispectral stuff
    b = zeros(MatSize,M); % Bispectrogram
end
t = zeros(1,M); % Time vector
err = zeros(Nseries,M); % Mean information
afft = zeros(Nseries,lim); % Welch power spectrum
spec = zeros(lim,M,Nseries); % Spectrogram
fft_coeffs = zeros(Nseries,lim); % Coeffs for slice
r = zeros(2,M); % Biphase proxy
Ntoss = 0; % Number of removed slices
% -----
%% Main loop
tic % Start stopwatch
fprintf(' Working... ')
ch1 = {'|','|','/','-','\','|','|','|'};
ch2 = {'-','|',':','|','|','|','|','|'};
for m=1:M
    % Loading bar!
    fprintf('\b\b\b\b\b\b\b\b\b3.0f%%s',100*m/M,ch1{mod(m,8)+1},ch2{mod(m,8)+1})

    t(m) = (m-1)*step/samprate; % Build time vector

    for k=1:Nseries
        Ym = Y(k,(1+(m-1)*step):(m-1)*step+subint)); % Select subinterval
        Ym = Ym(1:nf); % Take only what is needed for res
        if Detrend % Remove linear least-squares fit
            for j=1:Nseries
                Ym_old = Ym;
                %Y(j,:) = detrend.fast(Y(j,:)); % Detrend data
                s = (6/(nf*(nf^2-1)))*(2*sum((1:nf).*Ym) - sum(Ym)*(nf+1));
                % Here, I have just worked out the exact form
                Ym(j,:) = Ym(j,:) - s*(1:nf);
            end
        end
        Ym = win.*(Ym-mean(Ym)); % Remove DC offset, multiply by window

        p = fft(Ym)/nf; % Limit and normalize by vector length
        if SmoothFFT>1 % Might be useful?
            p = smooth(p,floor(SmoothFFT));
        end

        fft_coeffs(k,:) = p(1:lim); % Get interested parties
        err(k,m) = mean(abs(fft_coeffs(k,:))); % Find mean distribution

        if err(k,m)>=errlim
            fft_coeffs(k,:) = 0*fft_coeffs(k,:); % Blank if mean excessive
            Ntoss = Ntoss + 1;
        end
        %fft_coeffs = fft_coeffs.*(1-exp(-(lim:-1:1)/(lim/2))); % "Filter"
        spec(:,m,k) = fft_coeffs(k,:); % Build spectrogram
    end
end
afft = afft + abs(fft_coeffs); % Welch's power spectral density (PSD)

```

```

    if ~JustFFT
        [s,E12,E3] = cbispec(f,fft_coeffs,MatSize,Nseries); % See below

        e = e + s; % -> Bispectrum
        e12 = e12 + E12; % -> Expectation { Sums }
        e3 = e3 + E3; % -> "

        w = (abs(e).^2)./(e12.*e3+lilguy); % Bicoherence spectrum

        if Bispectro % Compute bispectrogram
            b(:,m) = b_vs_freq(s,Nseries);
        end
        r(1,m) = sum(e(:)); % Biphas modulation proxy
        r(2,m) = sum(w(:)); % Development of b^2
    end
end

fprintf('\b\b\n')
fprintf(' Complete! %d of %d intervals (%.0f%%) exceed given ...
    tolerance.\n',Ntoss,Nseries*M,Ntoss/M*100);
fprintf(' Process required %.5f seconds.\n',toc) % Done!
% -----
%% Output
info = struct('samprate',samprate,...
    'resolution',res,...
    'subint',subint,...
    'step',step,...
    'window',windoe,...
    'errlim',errlim,...
    'fscale',fscale,...
    'lilguy',lilguy,...
    'cmap',CMap,...
    'plottype',PlotType,...
    'autoscale',AutoScale,...
    'Nseries',Nseries,...
    'Nsig',[Nsig-Nextra Nextra],...
    'Nint',M,...
    'MatSize',MatSize);
info.options = varargin;
if Verbose
    disp(info); % Thinking about a "verbose" option...
end

W.info = info;
if Nseries>1
    W.fv = f(lim:end)/fscale;
    W.ff = f/fscale;
else
    W.fv = f(1:lim)/fscale;
end
W.ft = afft/M;
if ~JustFFT
    W.bs = e/M;
    W.bc = w;
    W.bp = r;
    if Bispectro
        W.bg = b;
    end
end
W.sg = spec;
W.tv = t;
W.er = err;

if PlotIt % Go to plotting tool if requested

```

```

        plot_results(W);
end
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Main cogwork
function [B,e12,e3] = cbispec(f,p,MatSize,Nseries)
% Produces bispectrum of signal
% Also calculates expectations
% -----

B = zeros(MatSize);
e12 = zeros(MatSize);
e3 = zeros(MatSize);
if Nseries==1 % Much more efficient for auto-bispectrum
    for k=1:floor(MatSize/2)+1
        for q=k:MatSize+1-k
            B(k,q) = p(q)*p(k)*conj(p(k+q-1));
            e12(k,q) = abs(p(q)*p(k))^2;
            e3(k,q) = abs(p(k+q-1))^2;
        end
    end
else % Must calculate all of bi-frequency space
    lim = (MatSize+1)/2;
    nyq = 2*f(end) - f(end-1);
    p = [conj(fliplr(p)) p(:,2:end)];
    if Nseries==2
        s = p(2,:); % If 2 signals, compute B_xyy
    else
        s = p(3,:); % Else do B_xyz
    end
    for k=1:MatSize % Nested for-loops aren't really the answer in MATLAB
        for q=1:MatSize
            if (f(k)+f(q))> -nyq && (f(k)+f(q))< nyq
                B(k,q) = p(1,q)*p(2,k)*conj(s(k+q-lim));
                e12(k,q) = abs(p(1,q)*p(2,k))^2;
                e3(k,q) = abs(s(k+q-lim))^2;
            end
        end
    end
    % But hey, we're done, aren't we?
end
% -----
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Filtering
function out = dealias(in)
% Just a standard low-pass filter w/ cut-off set at the Nyquist frequency
% It is impossible to dealias a signal after it has been digitized though
% -----
h = fdesign.lowpass('Fp,Fst,Ap,Ast',0.8,1,1,60);
H = design(h,'equiripple');
out = filtfilt(H.Numerator,1,in);
% -----

%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Bispectrogram
function b = b_vs_freq(B,Nseries)
% Sums bispectral features for one frequency
% Could be optimized, but I don't think it's worth it...
% -----
Bf = flipud(B);
MatSize = length(B);
b = zeros(1,MatSize);
offset = 0;
if Nseries>1
    offset = floor(MatSize/2);
end

```

```

for k=1:MatSize
    % Sum along axes
    b(k) = sum(diag(Bf,k-MatSize+offset)) + sum(B(k,:)) + sum(B(:,k));
    %b(k) = b(k)/(k+2*MatSize); % Normalize
end
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Built-in plotting tool
function plot_results(q,~)
% Plot/watch your results!
% Probably the most spaghetti-like code here...
% - - - - -
if ~isstruct(q) % See if subfunction is called from a closing figure...
    q = get(q, 'UserData'); % Grab data structure
end
try % Check for colormap errors
    cmapchk = eval(sprintf('%s(1)',q.info.cmap));
    if length(cmapchk)~=3
        warning('Bispec:cmapError','"%s" not a valid colormap...\nUsing "jet" ...
            instead.',q.info.cmap)
        q.info.cmap = 'jet';
    end
catch cmapbeef %#ok<*NASGU>
    warning('Bispec:cmapError','Colormap "%s" is unknown...\nUsing "jet" instead.',q.info.cmap)
    q.info.cmap = 'jet';
end

dim = q.info.Nseries; % Get Nseries

qwer = questdlg('What''s the word?','PlotMaker','Plot','Watch','Video','Plot');
pause(eps) % Query user about intentions...
Watch = false;
switch qwer
    case {'Video','Watch'}
        if isequal(qwer,'Watch')
            Watch = true;
        end
        if dim==1 % Extra options for auto-bicoherence
            qwerplot = questdlg('What kind?','MovieOptions','Full','Subplot','Full');
            pause(eps)
            if isempty(qwerplot); return; end
        end
        AviMake = true;
    case 'Plot'
        if dim==1 % See if they want to "explore"
            qwerexp = questdlg('What kind?','PlotOptions','Just plot','Explore','Just plot');
            pause(eps)
            if isequal(qwerexp,'Explore')
                H = figure;
                set(H,'position',[207 84 929 580]);
                set(H,'WindowButtonDownFcn',@explore_bispec)
                set(H,'KeyPressFcn',@switch_plot)
                set(H,'DeleteFcn',@plot_results)
                set(H,'UserData',q) % Set callback and load data
                explore_bispec(H,1)
                return
            elseif isempty(qwerexp)
                return
            end
        end
        AviMake = false;
    otherwise
        return % User must have closed window
end
pause(eps);

```

```

%% Video stuff
% I've included ways to do this in both R2009 and R2018...
% Should be as easy as commenting stuff out! =^]
if AviMake
    if ~Watch
        qwer = inputdlg({'Export as:', 'Frame rate'}, 'Exporter', 1, {date, '15'});
        pause(eps)
        if ~isempty(qwer)
            name = qwer{1};
            %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2009
            aviobj = avifile(name, 'compression', 'None', 'fps', str2double(qwer{2}));
            %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2018
            %v = VideoWriter([name '.avi'], 'Uncompressed AVI');
            %v.FrameRate = str2double(qwer{2});
            %open(v)
        else
            return % User has closed dialog
        end
    end

    h = figure; % No problems? Boom!
    set(h, 'position', [207 84 929 580]);
    set(h, 'DeleteFcn', @plot.results);
    set(h, 'KeyPressFcn', @switch_plot);
    set(h, 'UserData', q); % Set callback and load data
    OnePlot = true;
    if dim>1 % Compare Nseries to find options
        f = q.ff;
        X = -0.97*f(end); % X-position for "full" plot
    else
        f = q.fv;
        X = 0.03*f(end); % X-position for truncated domain
        if ~isequal(qwerplot, 'Full')
            OnePlot = false;
        end
    end
end

for m=1:q.info.Nint-1

    movie_frame(q, f, OnePlot, X, m) % See below
    q = get(h, 'UserData'); % Update in case of parameter change

    try; getframe; catch; return; end; %#ok<*CTCH, NOSEM>
    if ~Watch
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2009
        aviobj = addframe(aviobj, h);
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2018
        %writeVideo(v, currentFrame);
    end
    pause(eps);
end
if ~Watch
    msgbox('Data exported as video!', 'Success', 'help');
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2009
    v = close(aviobj); %#ok<NASGU>
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% R2018
    %close(v)
end
return % Pork chop sandwiches? Get the hell out of here!!!
end
[q, s, str, str1, str2] = scale_freq(q); % Check for scaling option
%% Standard plots
% - - - - -
% 1) Power spectra

```

```

h(1) = figure;
C = eval(sprintf('%s(256)',q.info.cmap)); % Load colormap
for k=1:dim
    semilogy(q.fv,abs(q.ft(k,:)), 'color',C(70+20*k,:), 'linewidth',2.5)
    hold on
end
hold off
legend(gca, 'toggle')
if ~q.info.autoscale; ylim([1e-3 1e0]); xlim([q.fv(1) q.fv(end)]); end
grid on
title('Power spectrum','fontsize',14,'fontweight','b')
plot_labels({str,'P^{1/2} (a.u.)'});
% - - - - -
% 2) Spectrogram
h(2) = figure;
for k=1:dim
    subplot(1,dim,k)
    imagesc(q.tv,q.fv,2*log10(abs(q.sg(:, :,k))));
    title('Spectrogram','fontsize',14,'fontweight','b')
    plot_labels({'Time (s)',str,'log_{10} P'});
    colormap(q.info.cmap);
    if ~q.info.autoscale; caxis([-6 -0]); end
end
% - - - - -
% 3) Utility plot
if isfield(q,'bc')
    h(3) = figure;
    set(h(3), 'KeyPressFcn',@switch_plot);
    set(h(3), 'DeleteFcn',@plot_results);
    set(h(3), 'UserData',q);
    set(h(3), 'Tag', 'utility');
    utility_plot(q);
end
% - - - - -
% 4) Bispectrogram
if isfield(q,'bg')
    h(4) = figure;
    if dim>1
        imagesc(q.tv,q.ff,2*log10(abs(q.bg)));
    else
        imagesc(q.tv,q.fv,2*log10(abs(q.bg)));
    end
    title('Bispectrogram','fontsize',14,'fontweight','b')
    plot_labels({'Time (s)',['\Sigma f (' s ')'],'log_{10} |\Sigma B|^2'});
    colormap(q.info.cmap);
    if ~q.info.autoscale; caxis([-8 -2]); end
end
% - - - - -
% Save utility
SaveYes = inputdlg('Save figures as?','SaveFigs',1,{[date '.png']});
pause(eps)
if ~isempty(SaveYes)
    for k=1:length(h)
        str = sprintf('%s_%d.png',SaveYes{1},k);
        saveas(h(k),str);
    end
end
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Backend for special figure
function utility_plot(q)
% Allows figure to control output
% - - - - -
[~,~,~,str1,str2] = scale_freq(q);
[dum,cbarstr] = whatplot(q,q.bc,q.bs);

```

```

if q.info.Nseries>1
    surf(q.ff,q.ff,dum);
    %imagesc(q.ff,q.ff,dum);
else
    surf(q.fv,q.fv,dum);
end
shading interp
plot_labels({str1,str2,cbarstr});
colormap(q.info.cmap);
%zlim([0 1]); caxis([0 1])
axis tight
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Plot labeller
function plot_labels(strings)
% Necessary to avoid oodles of repeated code!
% - - - - -
n = length(strings);
xlabel(strings{1},'fontsize',14,'fontweight','bold')
if n>1; ylabel(strings{2},'fontsize',14,'fontweight','bold'); end;
if n>2
    cbar = colorbar; % Quick/dirty way to add label to colorbar
    ylabel(cbar,strings{3},'fontsize',14,'fontweight','bold')
end
set(gca,'YDir','normal','fontsize',14); % YDir is crucial w/ @imagesc!
% - - - - -
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Select what to plot
function [dum,cbarstr] = whatplot(q,bc,bs) %#ok<*INUSD>
% Saves some redundancy
% - - - - -
guy = q.info.plotttype;
switch guy
    case 'bicoh'
        dum = bc;
        cbarstr = 'b^2(f_1,f_2)';
    case {'abs','real','imag','angle'}
        dum = eval(sprintf('%s(%s)',guy,'bs'));
        cbarstr = sprintf('%s% B(f_1,f_2)',upper(guy(1)),guy(2:end));
end
% - - - - -
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Frequency scaling
function [q,s,str,str1,str2] = scale_freq(q)
% Automatically scales axes to eliminate ugliness
% - - - - -
scale = log10(q.info.fscale);
tags = {'mHz','cHz','dHz','Hz',[],[],'kHz',[],[],'MHz',[],[],'GHz'};
s = 'Hz';
if mod(scale,1)~=0
    warning('Bispec:freqScale','\nFrequency scale not power of 10... Using "Hz".')
else
    if ~isempty(tags{scale+4})
        s = tags{scale+4}; % If listed above, use it!
    else
        warning('Bispec:freqScale','\nIncompatible frequency scale... Using "Hz".')
        q.fv = q.fv*10^scale; % Else, remove this oddity and use "Hz"!
        q.info.fscale = 10^scale;
        if isfield(q,'ff'); q.ff = q.ff*10^scale; end
    end
end
end
str = sprintf('Frequency (%s)',s);
str1 = sprintf('Frequency_1 (%s)',s);
str2 = sprintf('Frequency_2 (%s)',s);
% - - - - -
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```



```

%% Get single frame
function movie_frame(q,f,OnePlot,X,m)
% Inputs are kind of ugly/redudant, but it works wonderfully!
% - - - - -
[q,~,str,str1,str2] = scale_freq(q); % Get strings
dim = q.info.Nseries; % Number of time-series
Y = 0.94*f(end); % Y-position for text
C = eval(sprintf('%s(256)',q.info.cmap)); % For coloring the FFT trace

fft_coeffs = zeros(dim,length(q.ft)); % I really hate to put this here...
% but memory usage could be *worse*

for k=1:dim
    fft_coeffs(k,:) = q.sg(:,m,k).'; % Get coeffs from spectrogram
end

[s,E12,E3] = cbispec(f*q.info.fscale,fft_coeffs,q.info.MatSize,dim);
wee = (abs(s).^2)./(E12.*E3+q.info.lilguy); % Calculate bicoherence

if ~OnePlot; subplot(2,2,[1 3]); end % Subplot if asked
[dum,cbarstr] = whatplot(q,wee,s); % Get plottable
imagesc(f,f,dum) % Everyone's favorite plotter
colormap(q.info.cmap);
if ~q.info.autoscale && isequal(q.info.plotttype,'bicoh')
    caxis([0 1]);
end
plot_labels({str1,str2,cbarstr})
tstr = sprintf('t = %6.4g s',q.tv(m)); % Time @ slice
if dim==1
    ylim([0 f(end)/2]);
    text(X,Y/1.99,tstr,'fontsize',14,'fontweight','bold','color','white')
else
    text(X,Y,tstr,'fontsize',14,'fontweight','bold','color','white')
end
set(gca,'Tag','bicoh');

if ~OnePlot % Draw spectrogram w/ lines
    dt = q.info.subint/q.info.samprate; % Time across single subinterval
    subplot(2,2,2)
    imagesc(q.tv,f,2*log10(abs(q.sg)));
    line([q.tv(m),q.tv(m)], [0,f(end)], 'color','white')
    line([q.tv(m)+dt,q.tv(m)+dt], [0,f(end)], 'color','white')
    ylim([0 f(end)]);
    colormap(q.info.cmap);
    if ~q.info.autoscale; caxis([-6 0]); end
    plot_labels({'Time (s)',str,'log-10P'})
    set(gca,'Tag','spectro');

    subplot(2,2,4) % Power spectrum line-out
    semilogy(f,abs(fft_coeffs),'linewidth',2,'color',C(90,:))
    if ~q.info.autoscale; ylim([1e-3,1e0]); end
    xlim([0 f(end)]);
    grid on
    plot_labels({str,'P^{1/2} (a.u.)'})
    set(gca,'Tag','fft');
end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% Figure callback
function explore_bispec(obj,event)
% Subfunction to grab figure/explore
% - - - - -
q = get(obj,'UserData'); % "obj" is callback figure (=gcbf)
[q,~,str,str1,str2] = scale_freq(q); % Get strings and stuff
M = q.info.Nint; % Number of subintervals

if isempty(event) % "event" is used when figure is...

```

```

    m = event; % loaded, see above
else
    switch get(gca,'Tag')
    case 'spectro'
        P = get(gca,'CurrentPoint'); % Get point of mouse click
        if P(1,1)>=q.tv(1) && P(1,1)<=q.tv(end)
            [~,m] = min(abs(q.tv-P(1,1))); % Find closest point in time
        else
            return % Somewhere bad on figure
        end
    case 'fft'
        %[val,m] = max(q.sg(:,?));
        return
    case 'bicoH'
        return
    end
end
movie_frame(q,q.fv,false,0.03*q.fv(end),m)
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Plotting assist
function switch_plot(obj,event)
% For changing desired plottable
% - - - - -
press = event.Character;
switch press
    case {'B','A','R','I','P'}
        q = get(obj,'UserData');
        if isequal(press,'B')
            q.info.plotttype = 'bicoH';
        elseif isequal(press,'A')
            q.info.plotttype = 'abs';
        elseif isequal(press,'R')
            q.info.plotttype = 'real';
        elseif isequal(press,'I')
            q.info.plotttype = 'imag';
        elseif isequal(press,'P')
            q.info.plotttype = 'angle';
        end
        set(obj,'UserData',q); % Check if "utility" plot
        if isequal(get(obj,'Tag'),'utility')
            utility_plot(q)
        end
    otherwise
        return
end
% - - - - -
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Demonstration
function q = run_demo
% Nice for starting out, giving presentation, etc.
% - - - - -
pause(eps) % Eliminates some weird behavior
t = 0:1/100:100; % 100s time-vector sampled at 100 Hz
flip = @() mod(floor(100*rand),2); % Coin flip lambda

% Make 3 sinusoidal signals...
Afx = 6 + 4*(rand-1); % FM magnitude
Afy = 10 + 6*(rand-1);
Ff = 1/20; % FM frequency (kind of like LFO)
fx = 20 + (rand-1); % Carrier frequency
fy = 12 + (rand-1);
dfx = (Afx/fx)*sin(2*pi*t*Ff)./(t+eps); % Addition to sinusoidal argument
dfy = (Afy/fy)*cos(2*pi*t*Ff)./(t+eps);
x = sin(fx*2*pi*t.*(1+dfx)); % f1

```

```

y = sin(fy*2*pi*t.*(1+dfy)); % f2
z = sin(2*pi*t.*(fx*(1+dfx)+fy*(1+dfy))); % f1 + f2
u = sin(25*2*pi*t); % Unrelated oscillation

if ~(flip()&&flip())
    signal = x+y+(z+rand*u)+0.5*rand(1,length(t))-1; % 37.5% auto
    if flip()
        signal = [signal' signal' signal']; % 37.5% auto (full-frequency)
    end
else
    signal = [x' y' (z+rand*u)']; % 25% cross w/ 2 intersection
end

q = pplk.bispec(signal,100,0.2,512,128,...
    'dealias',false,...
    'window','gausswin',...
    'bispectro',true,...
    'plot',true,...
    'fscale',1,...
    'justfft',false,...
    'verbose',false,...
    'cmap','cmr');
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
%% Record in
function raw = record_demo
% Cool little extra
% - - - - -
pause(eps);
tf = 10;
fS = 44100/4;
for k=1:3
    fprintf('\n %s%d...',repmat(' ',1,4*(k-1)),4-k)
    pause(1);
end
fprintf('\n Recording now!')
y = wavrecord(10*fS,fS,1);
fprintf('\b\b\b\b\b...done.\n')

t = linspace(0,tf-1/fS,10*fS)';
raw = [t y]';

str = sprintf('%s%s','bic_audio',date);
qwer = inputdlg({'Save wav file?'},'SaveAs',1,{str});
pause(eps)
if ~isempty(qwer)
    if isequal(qwer{1},'0')
        save('raw_audio.mat','raw');
    else
        wavwrite(y,fS,qwer)
    end
end
end
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```